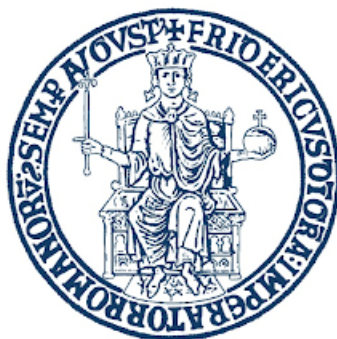




UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

**Progetto di Ingegneria del Software
A.A. 2025-2026**

Documentazione BugBoard26

11/07/2026

Petraccone Simone - N86005387
Picari Alessia - N86005131
Sorrentino Alessandro - N86004994

Contents

1	Introduzione	3
1.1	Scopo	3
1.2	Obiettivi	3
1.3	Glossario	3
2	Descrizione generale	5
2.1	Target utenti	5
2.2	Funzionalità	5
2.3	Caratteristiche degli utenti	5
2.4	Vincoli	6
2.4.1	Vincoli di architettura:	6
2.4.2	Vincoli tecnologici:	6
2.4.3	Vincoli di progettazione:	6
3	Requisiti funzionali	7
3.1	Utente generico	8
3.1.1	Autenticazione	8
3.1.2	Logout	8
3.1.3	Cambio password	9
3.1.4	Visualizzazione lista delle issue	9
3.1.5	Filtraggio e ordinamento lista issue	15
3.1.6	Esportazione lista issue	15
3.1.7	Visualizzazione singola issue	19
3.1.8	Visualizzazione cronologia issue	19
3.2	User e Admin	20
3.2.1	Segnalazione nuova issue	20
3.2.2	Creazione nuova tag	26
3.2.3	Visualizzazione notifiche	27
3.2.4	Modifica stato issue	28
3.3	Solo Admin	29
3.3.1	Creazione utenti	29
3.3.2	Assegnazione issue	30
3.3.3	Chiusura issue	31
3.4	Sistema	32
3.4.1	Aiuto nella scelta assegnazione issue	32
3.4.2	Notifica di risoluzione issue	32
3.4.3	Cronologia delle issue	32
4	Requisiti non funzionali e di sistema	32
4.1	Requisiti delle interfacce utente	32
4.2	Requisiti di performance	32
4.3	Requisiti di sicurezza	32

5	Architettura e design del sistema	33
5.1	Architettura proposta e criteri di design	33
5.2	Scelte tecnologiche	33
5.3	Client	34
5.4	Server	34
5.5	Containerizzazione	35
6	Design del software	36
6.1	Schema persistenza dati	36
6.2	Diagramma delle classi di design (UML Backend)	37
6.3	Scelte di software design	39
7	Deployment e infrastruttura cloud	40
7.1	Flusso delle comunicazioni e ciclo di vita della richiesta	41
7.2	Componenti dell'infrastruttura	41
7.3	Configurazione e variabili d'ambiente	41
7.3.1	Lato Frontend (Vercel)	41
7.3.2	Lato Backend (Render)	42
8	Processo di sviluppo e qualità del codice	42
8.1	Versioning e collaborazione	42
8.2	Continuous Integration e Continuous Deployment (CI/CD)	42
8.3	Report di qualità del codice	43
8.4	Artefatto software e repository link	44
9	Testing	44
9.1	Testing di unità (Backend)	44
9.1.1	Segnalazione di una issue (<code>createIssue</code>)	45
9.1.2	Assegnazione di una issue ad uno User (<code>assignUserToIssue</code>)	50
9.1.3	Cambio password utente (<code>changeUserPassword</code>)	54
9.2	Usability Testing	57
9.2.1	Survey a persone esterne	57
9.2.2	Modifiche effettuate	59

1 Introduzione

1.1 Scopo

Il presente documento definisce i requisiti funzionali e non-funzionali del sistema BugBoard26. Il sistema è destinato agli sviluppatori, ai tester e agli stakeholder dei progetti software dell'azienda. Vengono inoltre descritte le funzionalità del sistema, i ruoli e responsabilità degli utenti, i vincoli tecnologici, l'architettura utilizzata e l'attività di testing effettuata.

1.2 Obiettivi

Il software è una piattaforma web-based progettata per la segnalazione, l'assegnamento, la gestione e il tracciamento dello stato delle issue relative a progetti software.

1.3 Glossario

Di seguito vengono elencati i termini specifici utilizzati nel progetto, affiancati dal loro significato esplicito:

- **Issue:** una problematica o una richiesta che deve essere risolta/realizzata all'interno del sistema da un utente assegnato.
- **Tag:** un'etichetta che viene assegnata ad una issue, per chiarirne l'ambito (e.g. Frontend, Database).
- **Tipo (riferito ad issue):** ogni issue possiede un tipo, che può assumere uno tra i seguenti valori: *bug*, *question*, *documentation*, *feature*.
- **Bug:** malfunzionamento del software.
- **Question:** richiesta di chiarimenti riguardo una funzionalità del software.
- **Documentation:** segnalazione di problemi relativi alla documentazione.
- **Feature:** richiesta o suggerimento di nuove funzionalità.
- **Stato (riferito ad issue):** quando una issue viene creata assume lo stato TODO, che successivamente potrà essere impostato a IN_PROGRESS e RESOLVED. Oppure, in caso di issue ritenuta duplicata, potrà essere impostato (esclusivamente da un ADMIN) a CLOSED.
- **Utente:** utente generico che ha accesso alla piattaforma.
- **Lurker:** utente del sistema con *privilegi ridotti*.
- **User:** utente del sistema con *privilegi standard*.
- **Admin:** utente del sistema con *privilegi elevati*.

- **Segnalare (riferito ad issue):** l'azione della creazione di una nuova issue da parte di un utente (Admin o User).
- **Notifica:** messaggio che viene inviato automaticamente quando lo stato di una issue viene impostato a RESOLVED all'utente che ha segnalato la issue.
- **Privilegi ridotti:** l'utente che possiede tali privilegi può compiere le seguenti azioni:
 - visualizzare le issue presenti nel sistema (ovvero tutte le informazioni relative ad esse, e.g. titolo, descrizione, tipo, stato...).
 - esportare in formato CSV la lista di issue visualizzate.
 - visualizzare le Cronologia delle issue (tutte le azioni avvenute sulle issue con la rispettiva datazione e attore).
 - cambiare la password del proprio account.
- **Privilegi standard:** l'utente che possiede tali privilegi gode dei privilegi ridotti e può inoltre compiere le seguenti azioni:
 - segnalare una issue (inserendo i valori richiesti e quelli opzionali).
 - cambiare lo stato di una issue che gli è stata assegnata e si trova in uno stato modificabile (TODO e IN_PROGRESS).
 - visualizzare le issue da lui segnalate.
 - visualizzare le issue a lui assegnate.
 - visualizzare e segnare come lette le proprie notifiche.
- **Privilegi elevati:** l'utente che possiede tali privilegi gode dei privilegi standard e può inoltre compiere le seguenti azioni:
 - creare un nuovo utente (inserendo: username, mail, password e ruolo).
 - assegnare le issue (non ancora assegnate) ad un utente con privilegi standard.
 - chiudere una issue perché ritenuta duplicata.
- **Workload (Carico di lavoro):** viene calcolato in base al numero di issue assegnate, e non ancora risolte/chiusure, ad un determinato utente.
- **Frontend/Client:** l'interfaccia utente dell'applicazione.
- **Backend/Server:** l'infrastruttura lato server che gestisce la logica di business e i dati.
- **JWT (JSON Web Token):** standard utilizzato per la trasmissione sicura, tra client e server, delle informazioni di autenticazione dell'utente.

2 Descrizione generale

2.1 Target utenti

L'applicazione è accessibile solo tramite autenticazione, è inoltre presente un account di default (con ruolo Admin) da noi disposto in precedenza. La web-app è destinata principalmente agli impiegati e manager dell'azienda, ma è prevista la creazione di account "read-only" detti Lurker, destinata a persone esterne.

2.2 Funzionalità

Il sistema è stato realizzato per offrire funzionalità complete per il ciclo di vita di una issue: dalla sua creazione (con supporto per allegare immagini e tag) all'assegnazione guidata (in base al carico di lavoro degli utenti), fino alla risoluzione e/o chiusura. Viene incluso un sistema avanzato di filtraggio e ordinamento per la ricerca di issue, oltre alla possibilità di esportazione dei dati in formato CSV. È inoltre disponibile un sistema di tracciamento completo della cronologia delle modifiche delle issue. Per quanto riguarda la sicurezza, invece, il sistema è dotato di autenticazione tramite email e password, garantendo quindi una corretta gestione dei ruoli delle utenze.

2.3 Caratteristiche degli utenti

Il sistema è stato pensato per essere fruito da tre categorie di utenti: Admin, User e Lurker. Pensate per:

- Admin: utenti interni che si occupano di gestire l'assegnazione/chiusura delle issue e la creazione di nuove utenze.
- User: utenti interni addetti sia alla segnalazione di issue che alla risoluzione di quest'ultime.
- Lurker: utenti esterni, come gli stakeholder, che hanno la possibilità di visionare il lavoro effettuato sulle issue.

2.4 Vincoli

Di seguito sono elencati i vincoli imposti per la realizzazione del sistema richiesto:

2.4.1 Vincoli di architettura:

- L'applicazione deve essere progettata come un sistema distribuito composto da due macro-componenti strettamente indipendenti (Frontend e Backend). Ogni componente deve poter essere eseguito, modificato e sostituito in maniera autonoma, senza dipendenze dirette dalla struttura dell'altro.
- L'interazione tra Frontend (Client) e Backend (Server) deve avvenire esclusivamente attraverso la rete tramite API REST.
- Il Backend ha la responsabilità esclusiva della gestione centralizzata dello stato del sistema e della persistenza dei dati. È vietato l'utilizzo di servizi MBaaS esterni come Firebase o Supabase.

2.4.2 Vincoli tecnologici:

- È obbligatorio l'utilizzo di linguaggi di programmazione orientati agli oggetti per lo sviluppo.
- Il sistema deve essere accessibile via Internet tramite servizi di public Cloud Computing.

2.4.3 Vincoli di progettazione:

- È obbligatorio l'utilizzo di tool di CASE per supportare le attività di analisi e progettazione.
- Si richiede tassativamente di astrarre il design del software per favorire il riutilizzo del codice e la futura estendibilità del sistema.

3 Requisiti funzionali

Di seguito sono elencati i requisiti funzionali del sistema, con annessi gli use case diagram ed eventuali template di CockBurn:

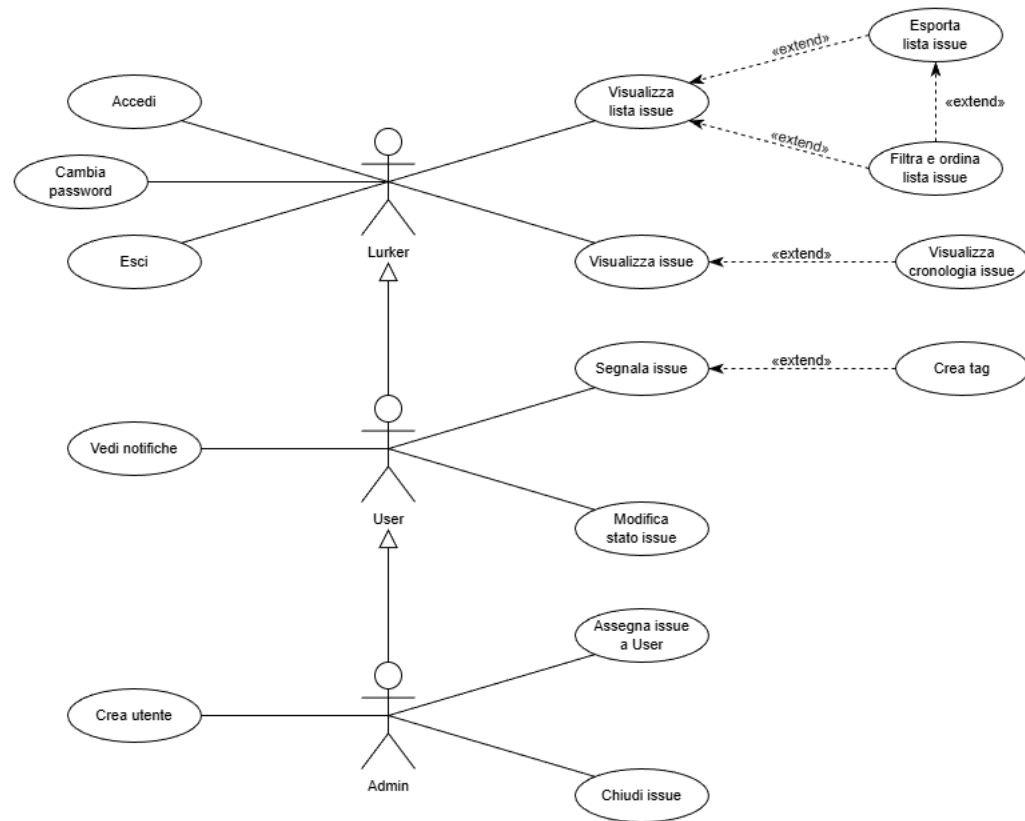


Figure 1: Use case diagram completo

3.1 Utente generico

Qualunque tipo di utente (Admin, User o Lurker) può svolgere queste azioni:

3.1.1 Autenticazione

Accedere al sistema inserendo le proprie credenziali (mail e password).

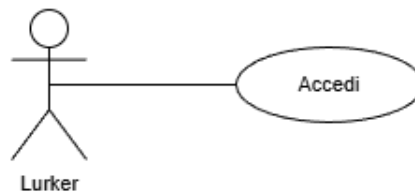


Figure 2: Use case diagram

3.1.2 Logout

Terminare la sessione e disconnettere il proprio account.

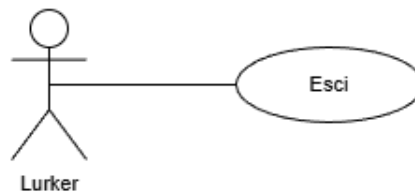


Figure 3: Use case diagram

3.1.3 Cambio password

Modificare la propria password, inserendo prima quella attuale.

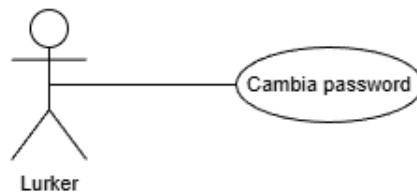


Figure 4: Use case diagram

3.1.4 Visualizzazione lista delle issue

Visualizzare l'elenco completo delle issue nel sistema.

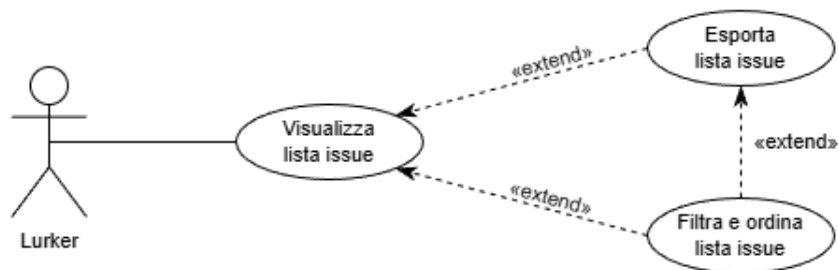


Figure 5: Use case diagram

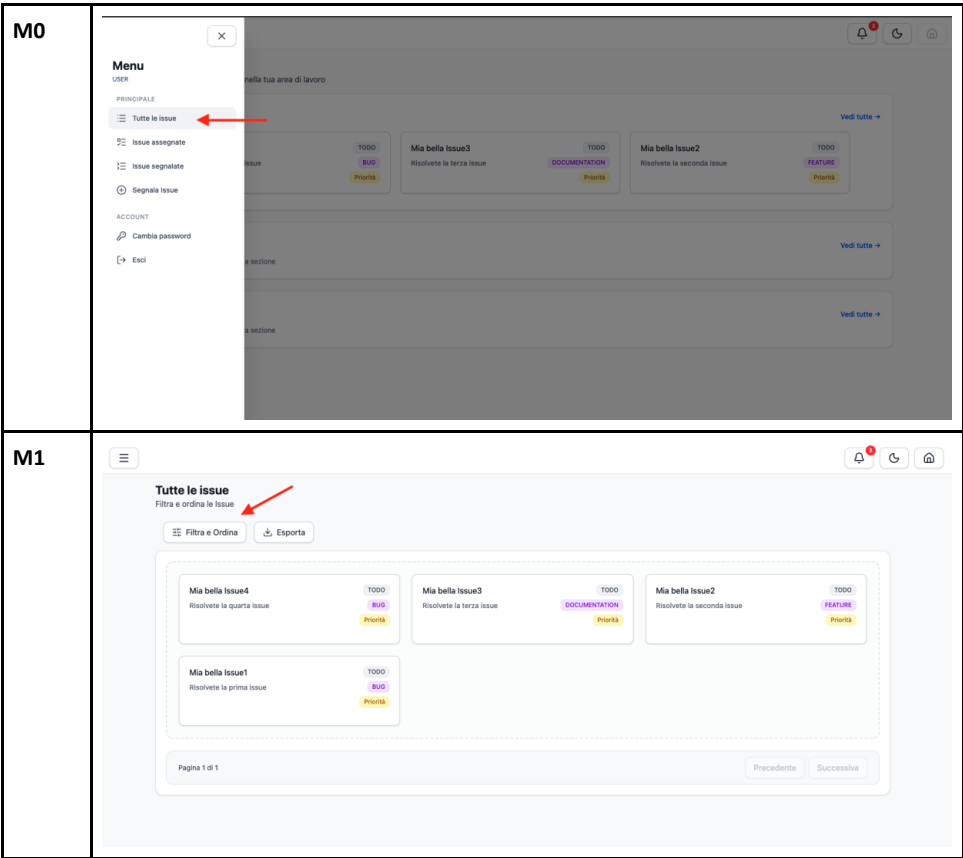
USE CASE #2	View Issue List		
Goal In Context	Visualizzare le issues.		
Preconditions	1. L'utente deve essersi autenticato (effettuato il login).		
Success End Condition	Vengono mostrate le issue secondo l'eventuale ordinamento e filtraggio impostato dall'utente.		
Failed End Condition	Lo use case non prevede alcuna condizione di fallimento.		
Primary Actor	Lurker		
Trigger	L'utente seleziona "Tutte le issue" dal menu principale.		
Main Scenario	Step n.	Lurker	System
	1	L'utente tramite il menu principale [M0] accede alla sezione dedicata alla visualizzazione delle Issue.	
	2		Mostra in [M1] o [M5].

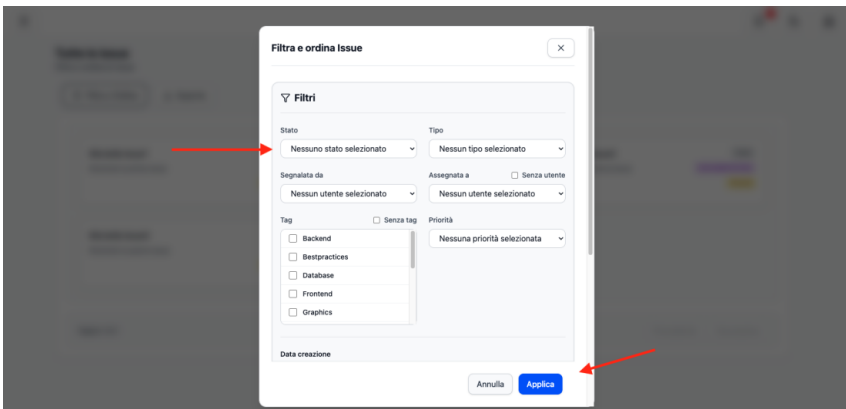
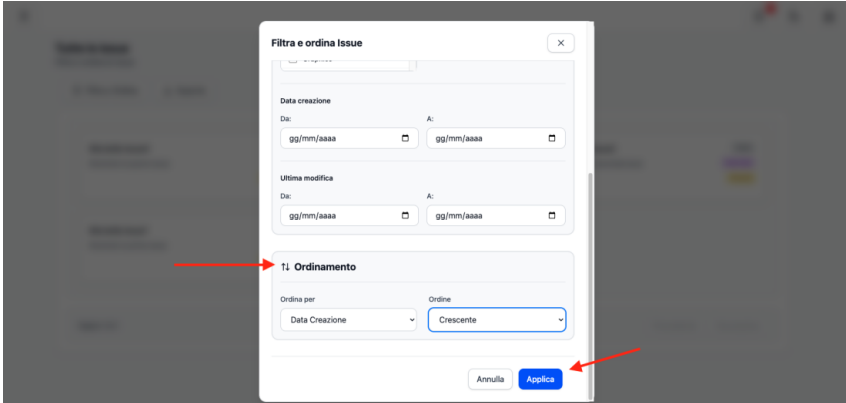
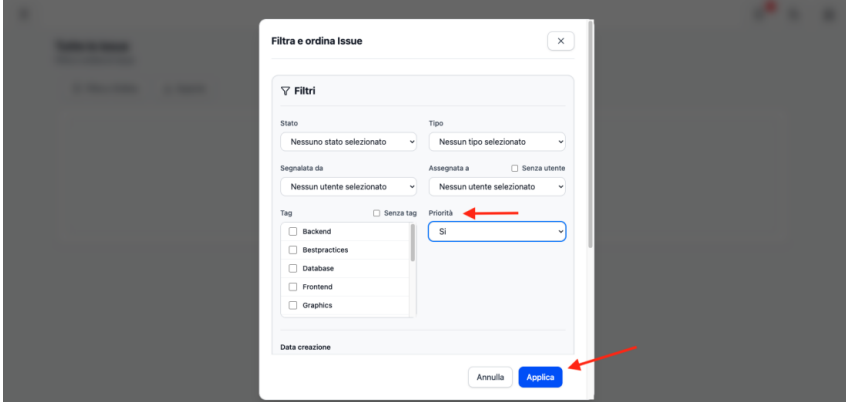
Extension A: Filtra	1A	L'utente clicca il tasto Filtra e Ordina [M1].	
	2A		Mostra [M2, 3].
	3A	Preme "Applica".	
	4A		Torna allo step 2 del Main Scenario.
Extension A1: Stato		Seleziona lo stato.	
			Torna a 2A.
Extension A2: Tipo		Seleziona il tipo.	
			Torna a 2A.
Extension A3: User		Seleziona lo User segnalatore.	

segnalatore			Torna a 2A.
Extension A4: User assegnato		Seleziona lo User assegnato (solo se la check-box "Senza utente" non è selezionata).	
			Torna a 2A.
Extension A5: Senza User assegnato		Seleziona la check-box "Senza utente".	
			Torna a 2A.
Extension A6: Tag		Seleziona i tag (solo se la check-box "Senza tag" non è selezionata).	
			Torna a 2A.
Extension A7: Senza tag		Seleziona la check-box "Senza tag".	
			Torna a 2A.
Extension A8: Priorità		Seleziona la priorità.	
			Torna a 2A.
Extension A9: Inizio data creazione		Seleziona una data.	
			Torna a 2A.
Extension A10: Fine data creazione		Seleziona una data.	
			Torna a 2A.
Extension A11: Inizio data ultima modifica		Seleziona una data.	
			Torna a 2A.
Extension A12: Fine data ultima modifica		Seleziona una data.	
			Torna a 2A.

Extension B: Ordina	1B	L'utente clicca il tasto Filtra e Ordina [M1].	
	2B		Mostra [M2, 3].
	3B	Seleziona il tipo di ordinamento.	
	4B	Seleziona ordine crescente o decrescente.	
	5B	Preme "Applica".	
	6B		Torna allo step 2 del Main Scenario.

Riferimenti UI



M2	
M3	
M4	

M5

Tutte le issue

Filtra e ordina le issue

Filtra e Ordina

Esporta

Nessuna issue trovata

Non ci sono risultati in questa sezione. Prova a modificare i filtri

3.1.5 Filtraggio e ordinamento lista issue

È possibile visualizzare una lista di issue effettuando la ricerca tramite filtri (su campi specifici) e ordinamento (crescente o decrescente).

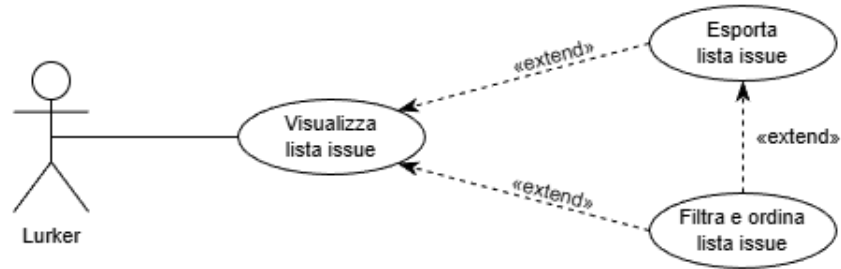


Figure 6: Use case diagram

3.1.6 Esportazione lista issue

Dopo aver visualizzato la lista di issue (eventualmente filtrata e ordinata), è possibile esportarla in formato CSV.

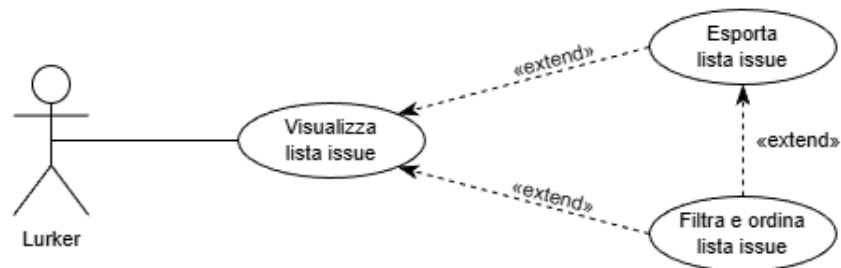


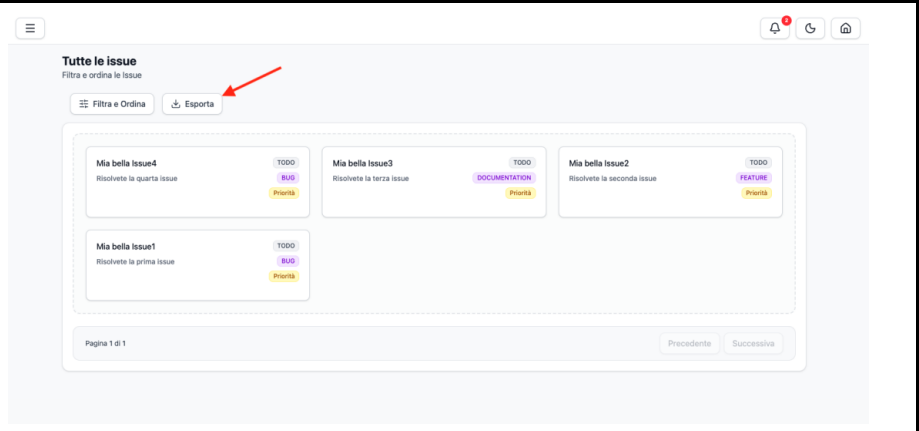
Figure 7: Use case diagram

USE CASE #3	Export issue list		
Goal In Context	L'utente vuole esportare la lista delle issue (opzionalmente filtrata e ordinata in precedenza), scegliendo il formato (CSV).		
Preconditions	1. L'utente deve essersi autenticato (effettuato il login). 2. L'utente deve trovarsi nella schermata di visualizzazione delle issue (opzionalmente ordinata e filtrata).		
Success End Condition	Il file richiesto dall'utente viene correttamente generato e scaricato sul dispositivo dal quale proviene la richiesta.		
Failed End Condition	Lo use case non prevede alcuna condizione di fallimento.		
Primary Actor	Lurker		
Trigger	L'utente clicca il pulsante "Esporta" nella schermata di visualizzazione delle issue.		
Main Scenario	Step n.	Lurker	System
	1	Richiede di esportare la lista delle issue cliccando sull'apposito pulsante [M0].	
	2		Mostra [M1].
	3	Seleziona il formato.	
	4		Recupera i dati delle issue attualmente visualizzate applicando i filtri e l'ordinamento eventualmente applicati con il livello di dettaglio selezionato.
	5		Formatta i dati, generando il file specifico richiesto.
	6		Avvia il download del file sul dispositivo dell'utente [M3].
Extension A: Livello di dettaglio	3A	Seleziona livello di dettaglio.	
	4A		Mostra [M2].

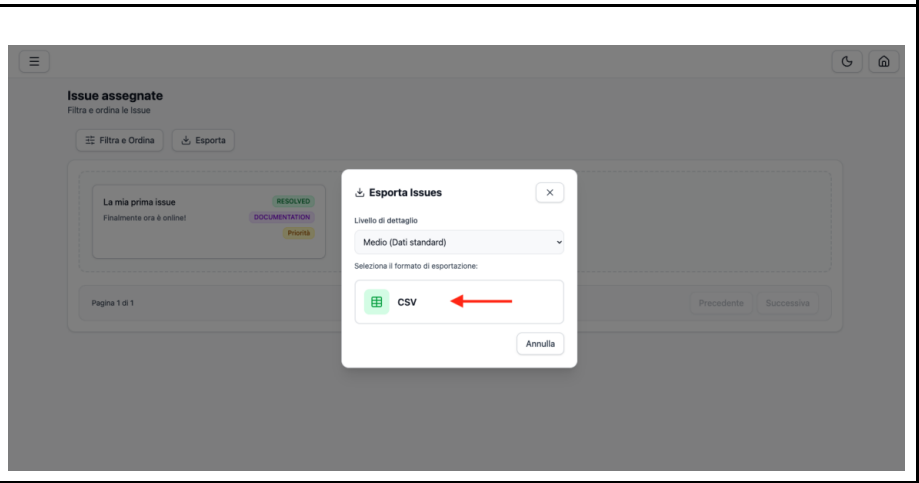
	5A	Seleziona l'opzione desiderata.	
	6A		Torna allo step 3 del Main Scenario.

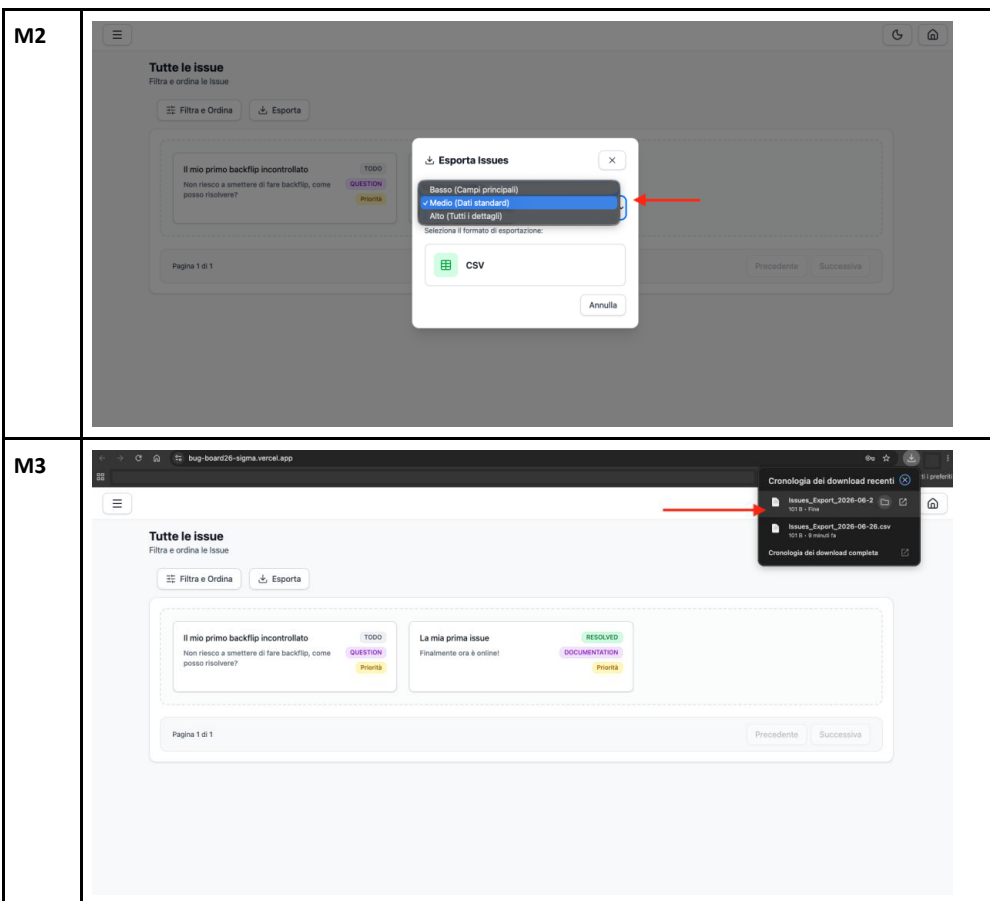
Riferimenti UI

M0



M1





3.1.7 Visualizzazione singola issue

Una volta visualizzato un elenco di issue, è possibile cliccare su una di esse per visualizzarla in maniera completa.



Figure 8: Use case diagram

3.1.8 Visualizzazione cronologia issue

Nella schermata di una issue è possibile accedere alla sua cronologia delle modifiche, così da poter visualizzare tutte le azioni effettuate.



Figure 9: Use case diagram

3.2 User e Admin

3.2.1 Segnalazione nuova issue

Specificando determinati campi, è possibile segnalare nuove issue all'interno del sistema.

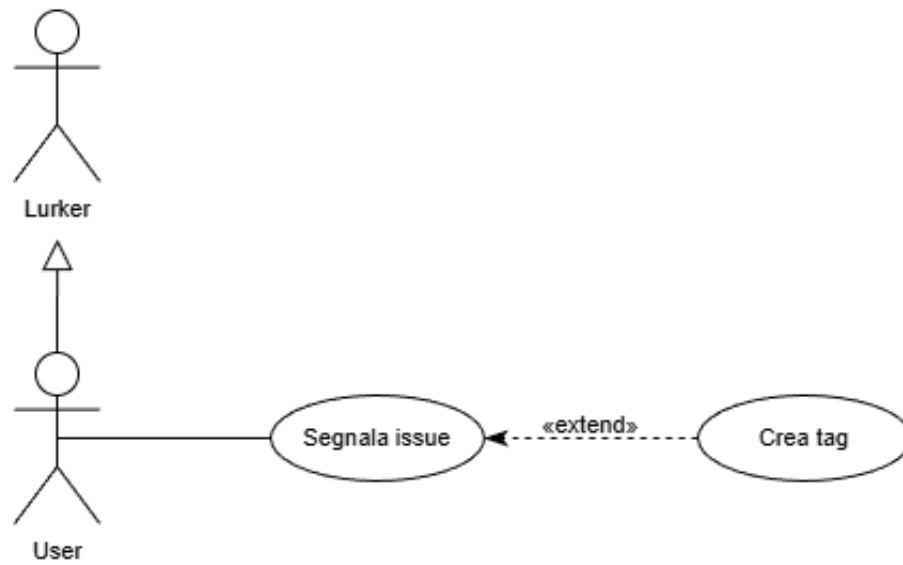


Figure 10: Use case diagram

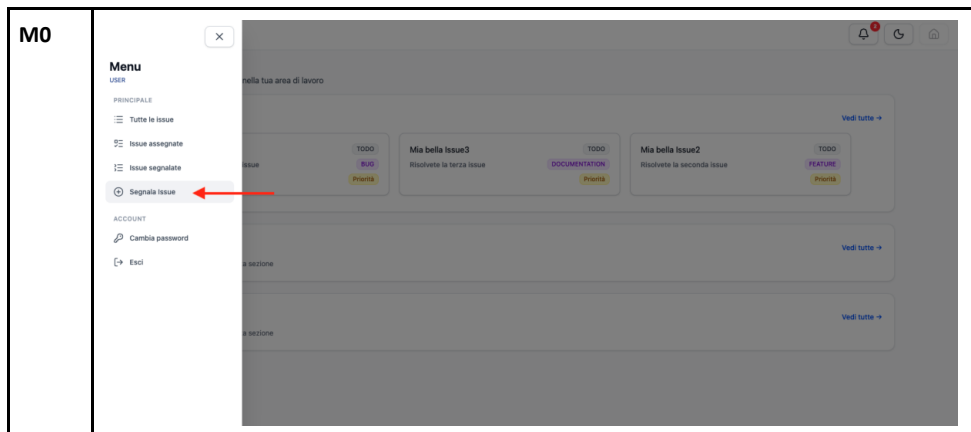
USE CASE #1	Report issue		
Goal In Context	Segnalare una nuova issue.		
Preconditions	1. L'utente deve essersi autenticato (effettuato il login).		
Success End Condition	La issue creata viene salvata con il testo e le opzioni specificate e messa nello stato TODO.		
Failed End Condition	Il sistema mostra una schermata di errore.		
Primary Actor	User		
Trigger	L'utente seleziona "Segnala Issue" dal menu principale.		
Main Scenario	Step n.	User	System
	1	Clicca su "Segnala issue" in [M0].	
	2		Mostra [M1].
	3	Inserisce il titolo.	
	4	Inserisce la descrizione.	
	5	Clicca sul menu a tendina "Tipo" in [M1] per specificare il tipo della issue che sta segnalando.	
	6		Mostra [M3].
	7	Seleziona l'opzione desiderata.	
	8	Clicca il tasto "Conferma" in [M1].	
	9		Mostra un messaggio di successo [M2].
Extension A: Aggiungi allegato	8A	Clicca sul tasto "Allega" per aggiungere un'immagine alla issue che sta segnalando.	
	9A		Mostra [M4].

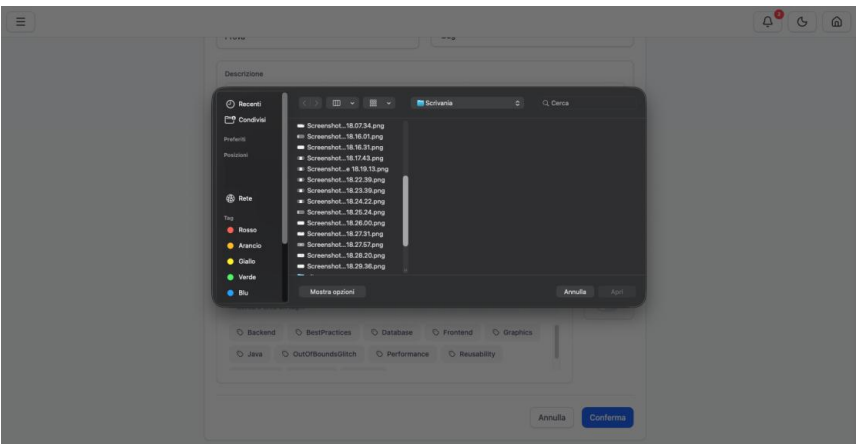
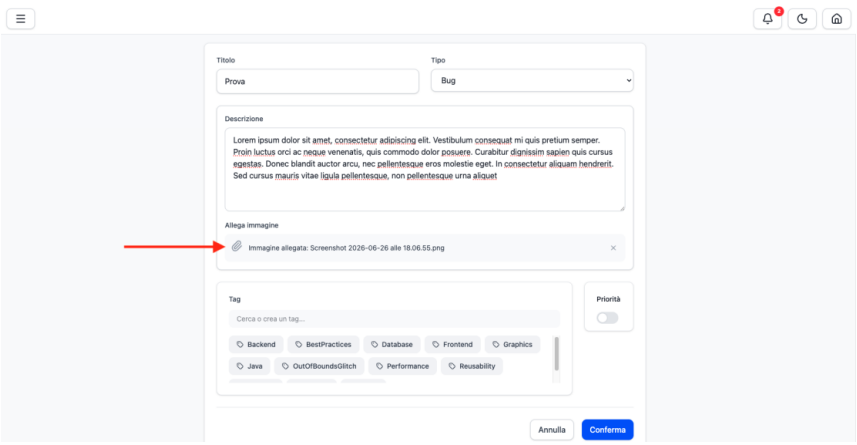
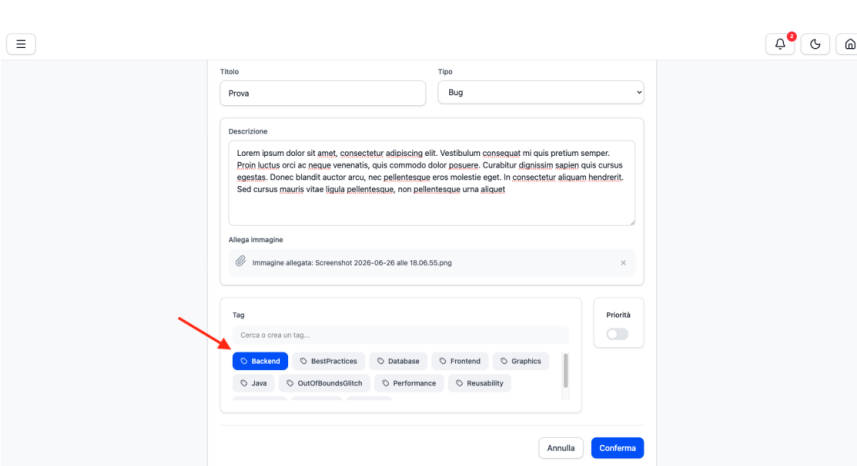
	10A	Seleziona l'immagine da allegare.	
	11A		Mostra [M5].
	12A		Ritorna allo step 8.

Extension B: Aggiungi tag	8B	Clicca sul tasto “Aggiungi Tag” per aggiungere un’etichetta alla issue che sta segnalando.	
	9B		Mostra [M6].
	10B	Seleziona una tra le opzioni disponibili [M6] o scrive una nuova tag [M7].	
	11B		Ritorna allo step 8.

Extension C: Specifica Priorità	8C	Clicca sullo switch “Priorità”.	
	9C		Mostra [M8].
	10C		Ritorna allo step 8.

Riferimenti UI



M4	
M5	
M6	

M7	Compila i campi e invia il modulo Titolo Prova Tipo Bug Descrizione Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum consequat mi quis pretium semper. Proin luctus orci ac neque venenatis, quis commodo dolor posuere. Curabitur dignissim sapien quis cursus egestas. Donec blandit auctor arcu, nec pellentesque eros molestie eget. In consectetur aliquam hendrerit. Sed cursus mauris vitae ligula pellentesque, non pellentesque urna aliquet Allega immagine Immagine allegata: Screenshot 2026-06-26 alle 18.06.55.png Tag React + Crea "React" Priorità ☐ Annulla Conferma
M8	Compila i campi e invia il modulo Titolo Prova Tipo Bug Descrizione Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vestibulum consequat mi quis pretium semper. Proin luctus orci ac neque venenatis, quis commodo dolor posuere. Curabitur dignissim sapien quis cursus egestas. Donec blandit auctor arcu, nec pellentesque eros molestie eget. In consectetur aliquam hendrerit. Sed cursus mauris vitae ligula pellentesque, non pellentesque urna aliquet Allega immagine Immagine allegata: Screenshot 2026-06-26 alle 18.06.55.png Tag Cerca o crea un tag... Java OutOfBoundsGitch Performance Reusability Security Testing UX/UI Priorità ☒ Annulla Conferma

3.2.2 Creazione nuova tag

Nella fase di segnalazione issue, è possibile selezionare tag esistenti oppure crearne di nuovi.

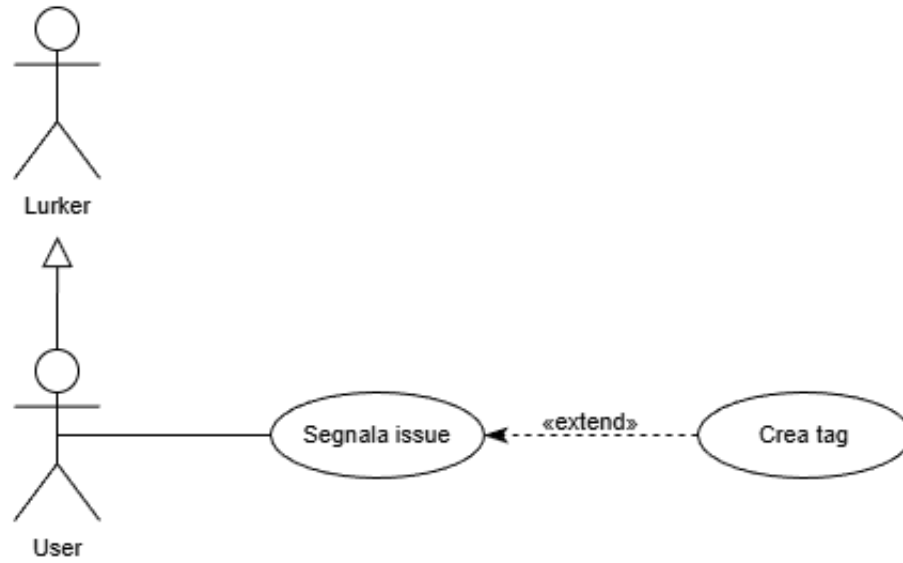


Figure 11: Use case diagram

3.2.3 Visualizzazione notifiche

Nella home è possibile visualizzare le notifiche ricevute, in merito alla risoluzione di issue segnalate.

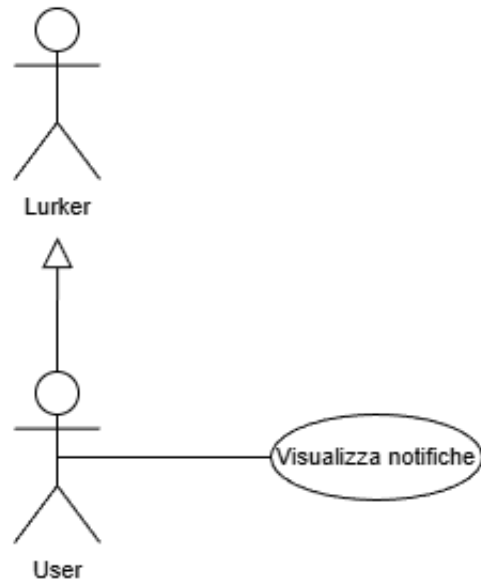


Figure 12: Use case diagram

3.2.4 Modifica stato issue

È possibile modificare le issue ricevute (assegnate da un Admin), cambiando il loro stato attuale.

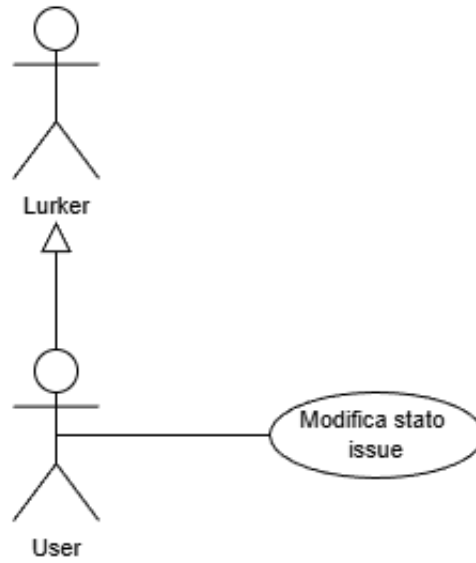


Figure 13: Use case diagram

3.3 Solo Admin

3.3.1 Creazione utenti

Avviene inserendo le credenziali di accesso e uno username (e.g. nome e cognome), specificando il ruolo del nuovo utente.

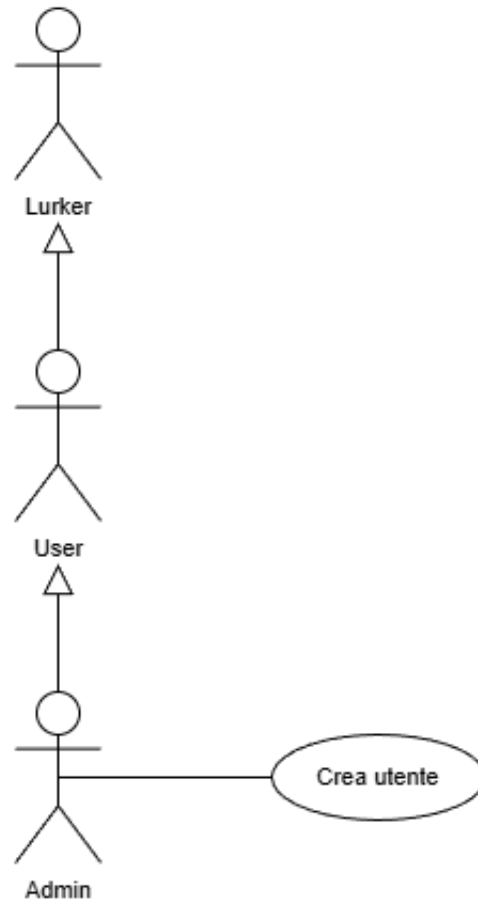


Figure 14: Use case diagram

3.3.2 Assegnazione issue

Dopo aver selezionato una issue non ancora assegnata, è possibile visualizzare l'elenco degli utenti in ordine di disponibilità. Così da poter eventualmente assegnare la issue con workload minore.

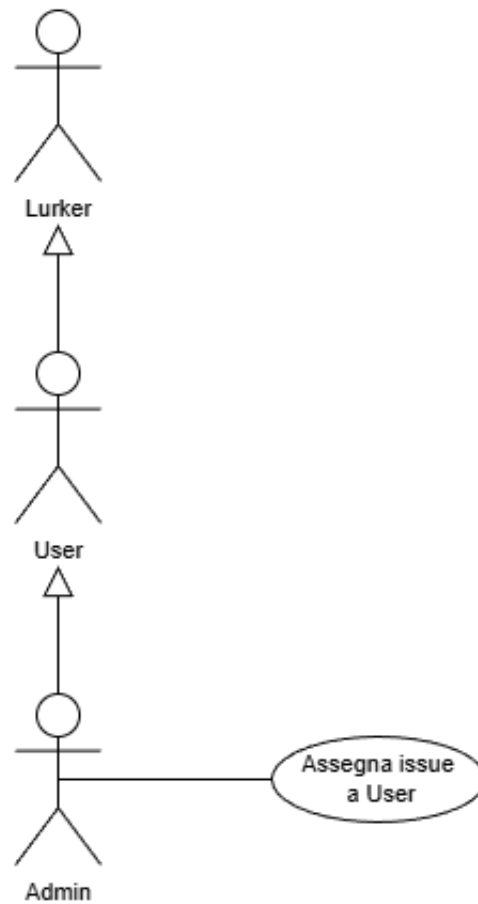


Figure 15: Use case diagram

3.3.3 Chiusura issue

Se viene rilevata una issue duplicata è possibile segnalarla come chiusa, in modo tale da evitare che due issue uguali possano essere risolte più volte.

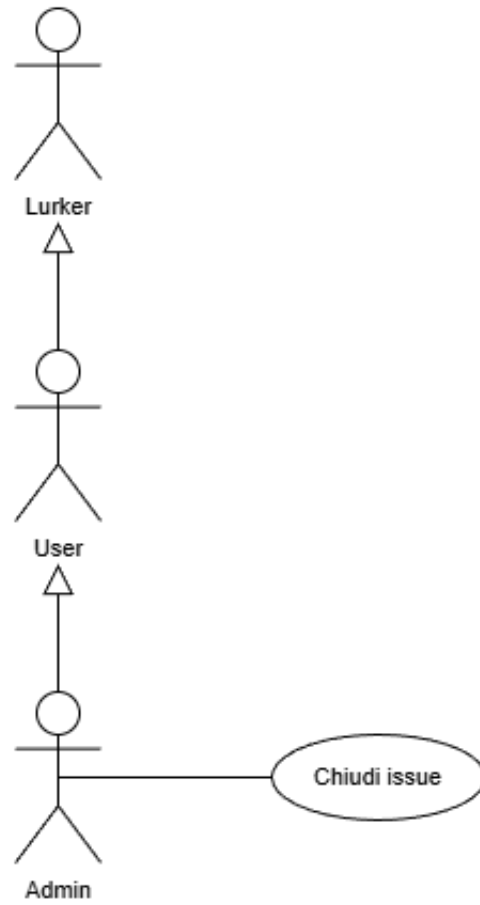


Figure 16: Use case diagram

3.4 Sistema

Il sistema deve offrire le seguenti funzioni agli utenti

3.4.1 Aiuto nella scelta assegnazione issue

Calcola il carico di lavoro assegnato ad ogni utente, così da poterli mostrare in ordine crescente di workload.

3.4.2 Notifica di risoluzione issue

Notifica l'utente che ha segnalato una issue quando viene risolta.

3.4.3 Cronologia delle issue

Ogni qualvolta viene effettuata una modifica allo stato di una issue (compresa la sua creazione), memorizza tale informazione all'interno del Database.

4 Requisiti non funzionali e di sistema

4.1 Requisiti delle interfacce utente

L'interfaccia web deve essere responsive, garantendo una corretta visualizzazione sia su dispositivi desktop che mobile. La UI deve seguire i principi del Material Design per offrire un'esperienza utente coerente, prevedibile e accessibile, con feedback visivi chiari per ogni azione.

4.2 Requisiti di performance

Il sistema deve garantire tempi di risposta rapidi: le operazioni di lettura (e.g. caricamento della lista issue) devono completarsi in meno di 2 secondi sotto condizioni di normale carico. L'architettura REST deve supportare richieste concorrenti senza degrado delle prestazioni.

4.3 Requisiti di sicurezza

Il sistema deve assicurare l'integrità e la riservatezza dei dati:

- Autenticazione: il token JWT (con scadenza definita) garantisce una comunicazione protetta tra client e server.
- Gestione password: le password nel Database sono salvate con una funzione di hash per garantire che le password non vengano mai salvate in chiaro nel Database.
- Autorizzazione: le API lato backend verificano le autorizzazioni dell'utente richiedente, così da impedirgli l'accesso a determinate operazioni.

5 Architettura e design del sistema

5.1 Architettura proposta e criteri di design

L'architettura del sistema è stata realizzata seguendo il modello Client-Server distribuito, così come richiesto nei vincoli di progetto, garantendo quindi la totale indipendenza tra la logica di presentazione e la logica di business. Il sistema è suddiviso in due macro componenti autonomi: il Frontend e il Backend. Tra i due avviene una comunicazione di tipo stateless, esclusivamente tramite chiamate di rete esponendo un'interfaccia REST API. Di seguito sono riportati i criteri di design adottati:

- **Separation of Concerns (SoC):** la netta divisione tra client e server garantisce che il Frontend si occupi unicamente della gestione della UI e dell'interazione con l'utente, mentre il Backend assume la responsabilità esclusiva della gestione dello stato, della logica applicativa e della persistenza dei dati.
- **Layered architecture:** il server è stato progettato seguendo il pattern a livelli Controller-Service-Repository. Questa scelta è stata attuata per favorire l'astrazione e il riutilizzo del codice. Il livello di interfaccia utente REST API (Controller) è disaccoppiato dall'accesso ai dati (Repository) tramite il livello intermedio (Service) che contiene la logica di business. In questo modo, eventuali modifiche al Database non influiscono i Controller esposti.
- **Inversion of Control (IoC) e Dependency injection:** per garantire un basso accoppiamento tra le classi, il design sfrutta l'iniezione delle dipendenze. Questo facilita la manutenibilità e la testabilità del sistema, permettendo di sostituire facilmente l'implementazione di un componente, senza alterare chi ne fa uso.

5.2 Scelte tecnologiche

Per soddisfare i requisiti architetturali e il vincolo riguardante l'utilizzo del paradigma Object-Oriented, sono state utilizzate le seguenti tecnologie:

- **Backend: Java e framework Spring Boot:** Java è il linguaggio Object-Oriented per eccellenza ed è considerato uno standard per lo sviluppo di software aziendali. Spring Boot è invece molto utile perché implementa nativamente IoC e Dependency injection. Inoltre, l'ecosistema Spring (tramite Spring Data JPA e Hibernate) permette di mappare in modo efficiente le classi di dominio (ossia le entità del sistema), garantendo una persistenza dei dati robusta e completamente gestita internamente al server, così da non ricorrere a servizi MBaaS.
- **Frontend: libreria React:** permette di creare una Single Page Application (SPA) modulare basata sul concetto di "componente". Questa

scelta favorisce la riusabilità del codice lato client. Inoltre, l'uso del Virtual DOM garantisce alte prestazioni nei rendering di interfaccia, offrendo un'esperienza utente fluida. Il Frontend risulta quindi totalmente indifferente rispetto all'implementazione del Backend, comunicando con esso unicamente tramite protocollo HTTP.

- **Peristenza dei dati: PostgreSQL:** è uno dei Database relazioni open-source più avanzati e robusti al mondo, in grado di garantire le proprietà ACID (Atomicity, Consistency, Isolation, Durability). Assicura che le transazioni critiche (e.g. cambio password) avvengano in totale sicurezza e senza perdite o corruzione di dati. Esso è perfetto per il paradigma Object-Oriented perché è facile mappare le classi Java nelle tabelle SQL.

5.3 Client

A livello strutturale, l'applicativo lato client è stato sviluppato seguendo il paradigma Component based, organizzando le risorse in una gerarchia piatta, così da favorire la rapidità di sviluppo e la facile reperibilità del codice. L'intera logica di presentazione e le viste principali sono aggregate all'interno di una directory centralizzata (`src/components`). Al suo interno convivono sia i componenti strutturali che fungono da pagine intere (e.g. `HomePage.jsx`, `LoginPage.jsx`), sia micro-componenti riutilizzabili dell'interfaccia (e.g. `CustomButton.jsx`, `IssueCard.jsx`).

La comunicazione con il server è stata disaccoppiata dai componenti visivi tramite un modulo di configurazione dedicato (`apiConfig.js`). Tale file agisce come da ponte per le chiamate di rete asincrone, centralizzando gli endpoint e le configurazioni delle API REST. Questa scelta progettuale rispetta il principio DRY (Don't Repeat Yourself), permettendo di modificare gli indirizzi del Backend in un unico punto dell'applicazione, senza dover alterare i singoli componenti React.

Infine il componente radice (`App.jsx`) svolge il ruolo di orchestrare il sistema e funge da contenitore principale per la Single Page Application.

5.4 Server

L'infrastruttura lato server, implementata tramite il framework Spring Boot, adotta un pattern ad architettura a strati per garantire la massima coesione interna dei moduli e un basso accoppiamento.

Il livello più esterno è rappresentato dal Controller layer, che svolge il ruolo di intermediario tra Frontend e Backend. Esso ha il compito di intercettare le richieste HTTP, validare la correttezza dei payload e restituire risposte.

Subito al di sotto si colloca il Service layer, che costituisce il nucleo operativo dell'applicazione. In questo strato è contenuta la logica di business, che comprende le regole di assegnazione issue, la gestione della cronologia delle issue e molteplici operazioni complesse.

Infine, la comunicazione con il Database PostgreSQL è delegata al Repository

layer. Quest'ultimo livello, tramite Spring Data JPA e Hibernate, permette di tradurre le operazioni sulle entità Java in query SQL, isolando così i livelli superiori dai dettagli implementativi legati alla persistenza fisica dei dati.

5.5 Containerizzazione

Il processo di containerizzazione si è concentrato sulla componente Backend, per la quale è stato realizzato un Dockerfile basato su una strategia di Multi stage build. Questo consente di isolare l'ambiente di compilazione (Maven) da quello di esecuzione (JRE), generando un'immagine finale leggera e sicura, pronta per essere rilasciata nel cloud.

Al fine di gestire la massima flessibilità e disaccoppiamento tra il codice e l'infrastruttura, è stato deciso di configurare i servizi (in particolare la connessione al database di PostgreSQL) in modo dinamico, attraverso l'uso di variabili di ambiente. Nel file `application.properties` il parametro di connessione al Database è stato mappato come `spring.datasource.url=DB_URL`, così da permettere l'utilizzo del medesimo codice sorgente sia in ambiente di sviluppo locale (iniettando la variabile tramite l'IDE IntelliJ), sia nell'ambiente di produzione esposto online (configurando la variabile direttamente sulla piattaforma Render).

Contemporaneamente, la componente Frontend non ha richiesto containerizzazione esplicita, in quanto è stata ottimizzata per il deploy nativo sulla piattaforma Vercel. L'integrazione diretta con il repository GitHub permette build automatizzate, mentre la comunicazione di rete con il Backend è gestita iniettando l'endpoint del server tramite la variabile d'ambiente `VITE_API_URL` durante la fase di build.

6 Design del software

6.1 Schema persistenza dati

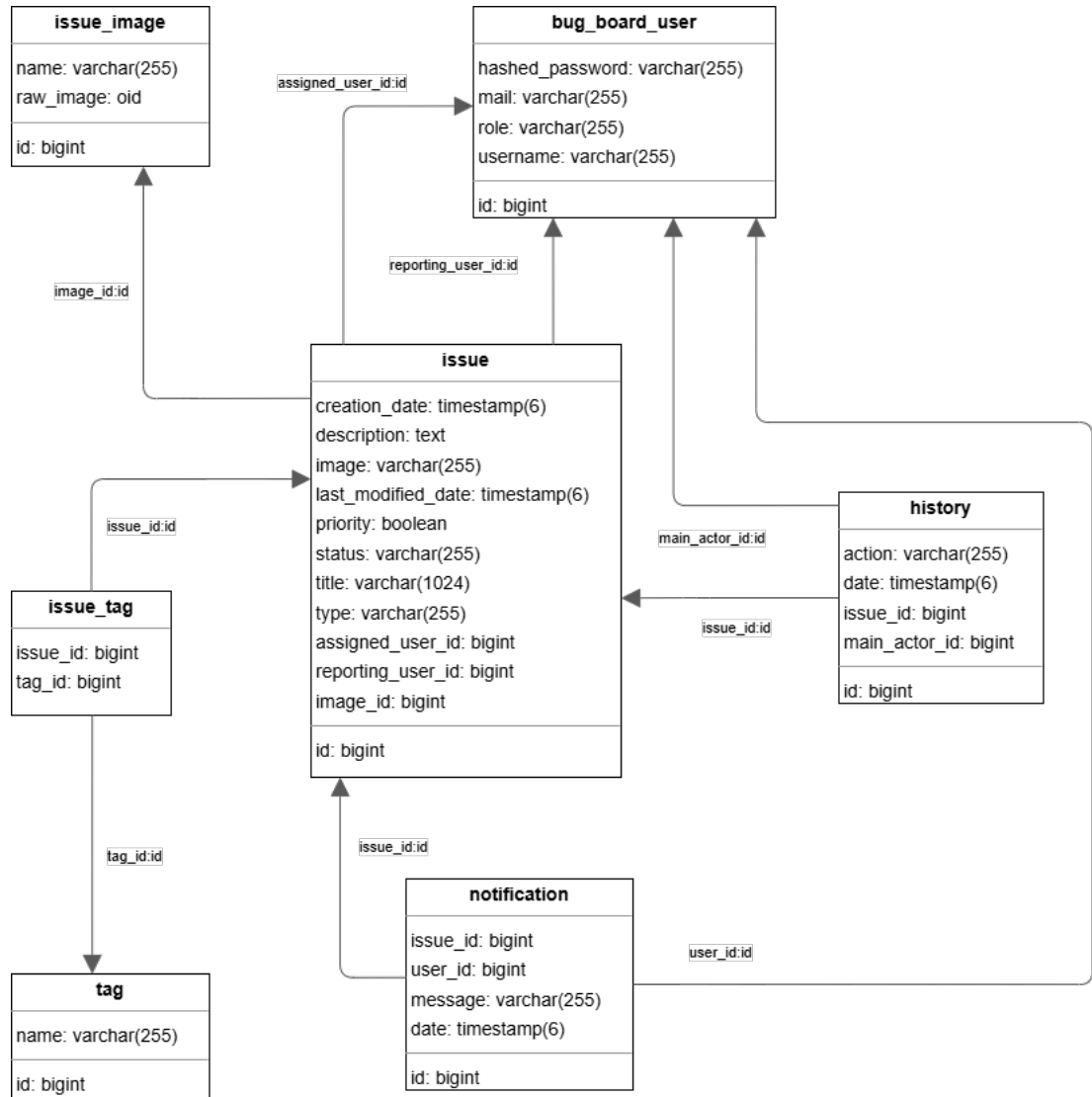


Figure 17: Diagramma ER

6.2 Diagramma delle classi di design (UML Backend)

Per comodità, il diagramma è stato diviso in due parti, la prima contenente i Service e i relativi Controller delle classi Issue, Notification e History. Mentre il secondo contenente i Service e i relativi controller delle classi Tag, User e Authentication.

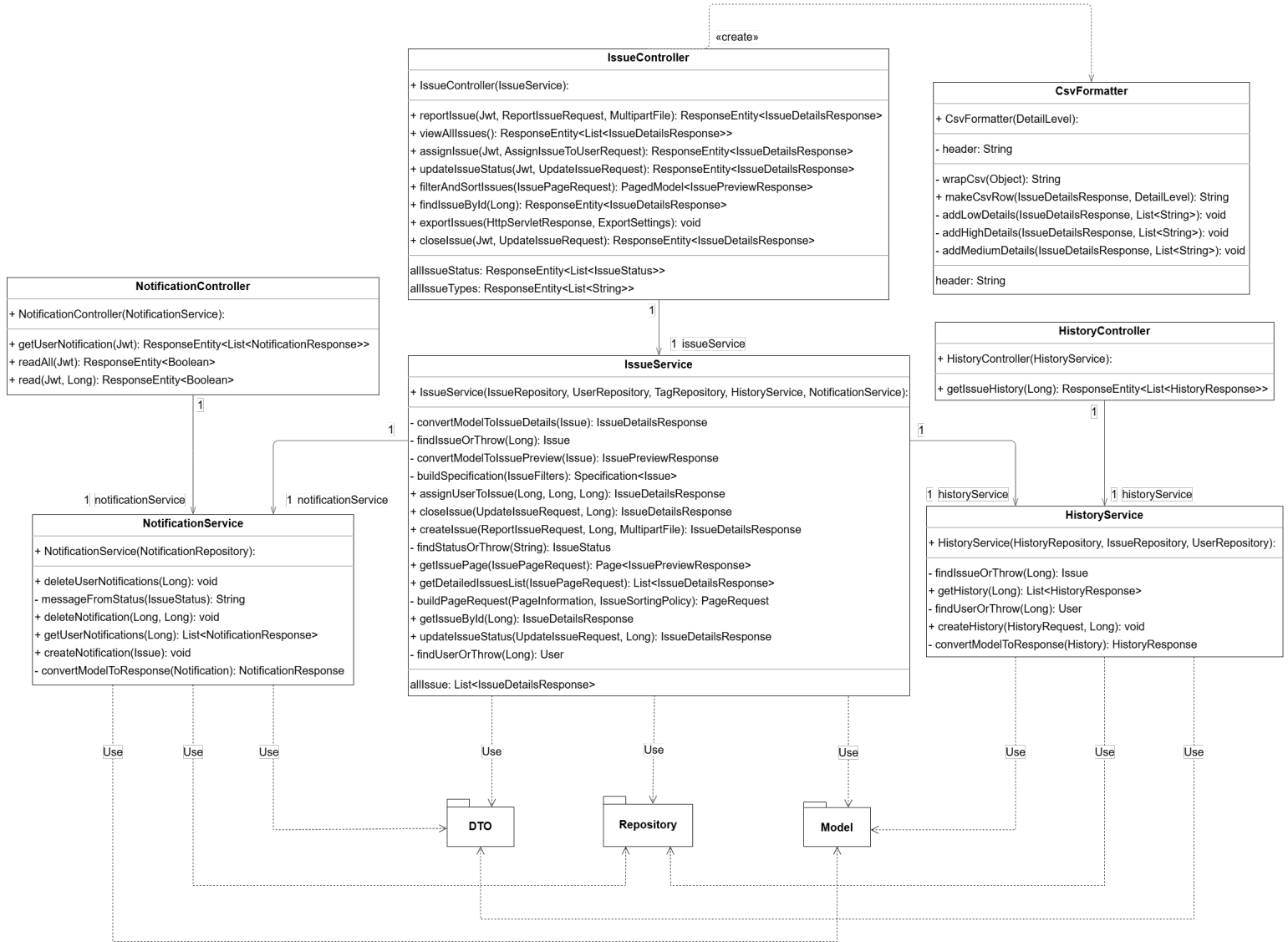


Figure 18: Diagramma UML parte 1

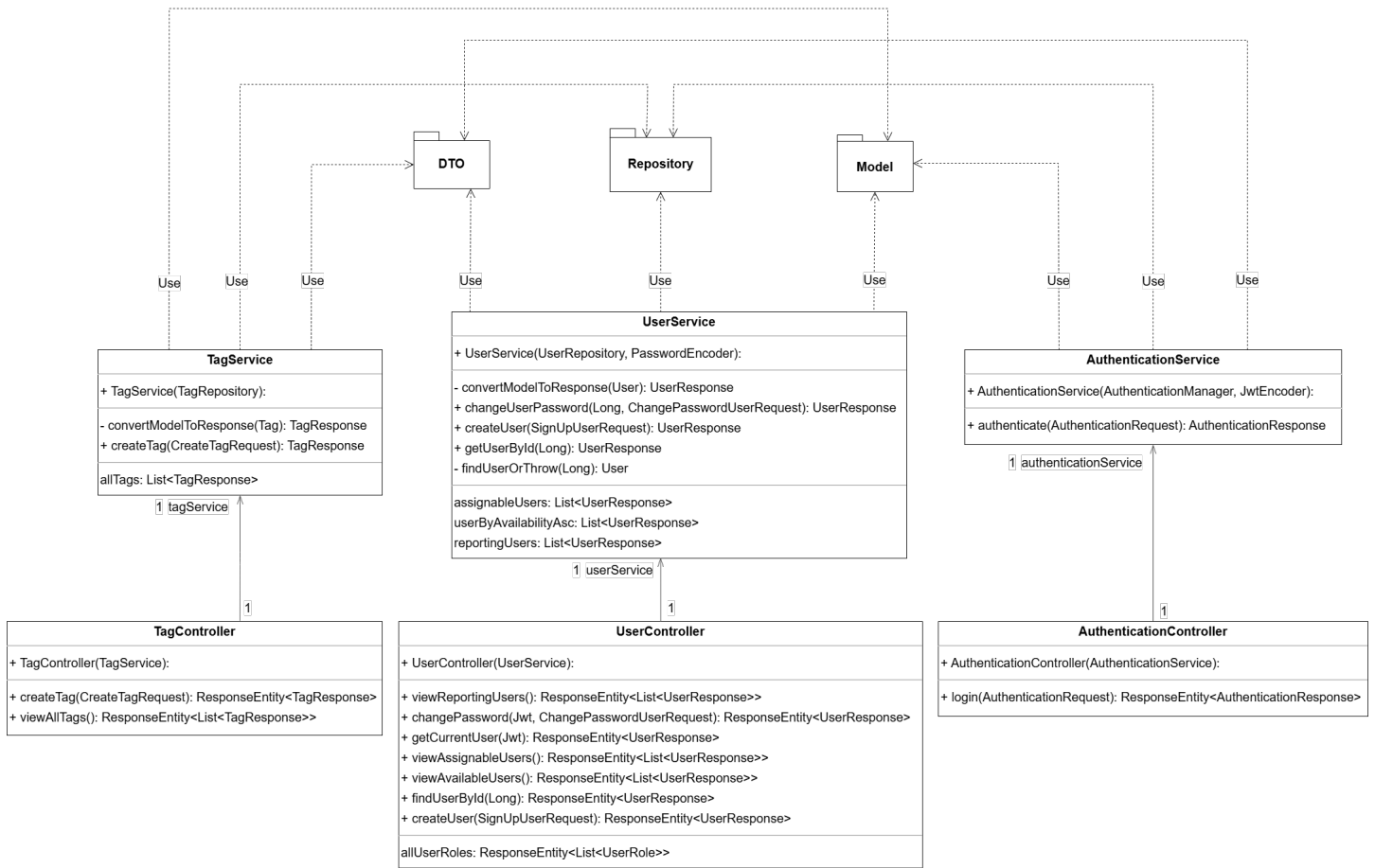


Figure 19: Diagramma UML parte 2

Nota: per semplicità non sono state rappresentate le classi all'interno dei Package DTO, Model e Repository. Ma risulta semplice riconoscere quali classi utilizza ciascun Service, vedendo quali parametri presentano i suoi metodi e costruttore.

6.3 Scelte di software design

Per garantire un'elevata qualità, manutenibilità ed estensibilità del codice, l'architettura del sistema implementa diversi design pattern:

- **Dependency Injection:** come anticipato nella sezione relativa ai criteri architetturici (IoC), a livello di implementazione del codice è stato fatto uso del pattern Dependency injection. Invece di istanziare esplicitamente gli oggetti (e.g. tramite `new`), il framework Spring Boot inietta a runtime le dipendenze necessarie (come i Service o i Repository) all'interno delle classi che ne fanno uso. Questo pattern applicato, garantisce basso accoppiamento delle classi rendendo i moduli altamente testabili.
- **Repository Pattern:** l'accesso al Database PostgreSQL è stato astratto interamente tramite le interfacce di Spring Data JPA. Questo pattern garantisce l'isolamento della logica di persistenza dal resto dell'applicazione, permettendo al livello Service di manipolare i dati vedendoli come oggetti Java.
- **Data Transfer Object (DTO):** per la comunicazione tra client e server non vengono mai esposte direttamente le entità del Database. Vengono invece utilizzate classi DTO intermedie per modellare i payload delle richieste e delle risposte. Questa scelta protegge dati sensibili (e.g. hash delle password) ed evita il sovraccarico di informazioni restituite nel JSON di risposta.
- **Model View Controller (MVC):** utilizzato per separare le responsabilità dei ruoli all'interno del Backend.
Il Model definisce e gestisce la struttura dei dati dell'applicazione (Entità e DTO).
La View corrisponde alla rappresentazione dei dati (il payload) formattata in JSON che il server restituisce al client.
Il Controller infine, svolge il compito di intercettare le richieste HTTP in ingresso dal Frontend e delegare l'esecuzione della relativa logica di business ai moduli Service.
- **Pattern Builder:** è stato fatto grande utilizzo del pattern Builder, implementato in modo dichiarativo tramite l'annotazione `@Builder` della libreria Lombok. Questo approccio ha permesso di istanziare oggetti complessi e DTO in modo fluido, leggibile e immutabile.

7 Deployment e infrastruttura cloud

Il progetto è strutturato su un'architettura cloud moderna a servizi separati, che garantisce scalabilità e un flusso di aggiornamento completamente automatizzato.

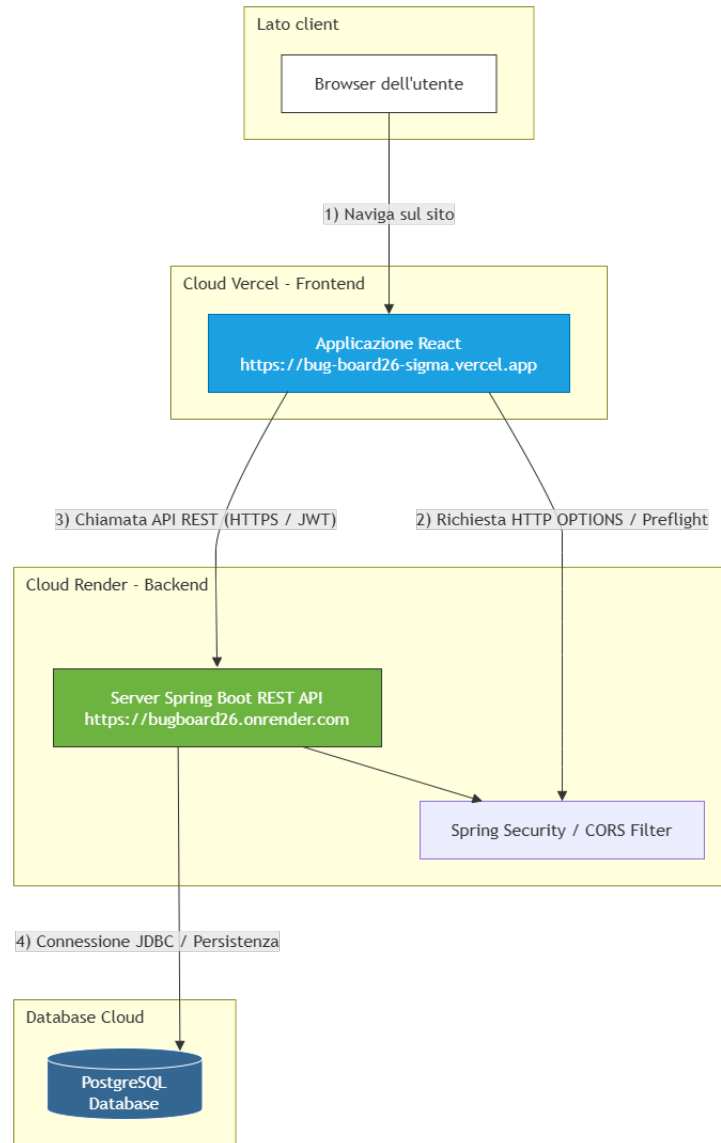


Figure 20: Diagramma di deployment e flusso delle comunicazioni HTTP/JDBC dell'infrastruttura cloud.

7.1 Flusso delle comunicazioni e ciclo di vita della richiesta

- **Fase 1 (Navigazione):** L'utente accede tramite browser all'applicazione React, servita dalla CDN globale di Vercel.
- **Fase 2 (Controllo sicurezza):** Prima di inviare dati sensibili, il browser effettua una richiesta HTTP OPTIONS verso il backend su Render. Il filtro CORS di Spring Security intercetta la chiamata e valida l'origine (Vercel), autorizzando la transazione.
- **Fase 3 (Chiamata API autenticata):** Il frontend invia la richiesta REST effettiva (e.g. il login) allegando il token JWT nell'header di autorizzazione. Spring Boot elabora la logica di business.
- **Fase 4 (Persistenza dati):** Il server Spring Boot comunica con l'istanza cloud del Database PostgreSQL tramite connessione JDBC per leggere o scrivere i dati richiesti, restituendo infine la risposta in formato JSON al client.

7.2 Componenti dell'infrastruttura

Il sistema, in ambiente di produzione, si appoggia ai seguenti servizi gratuiti/-PaaS:

- **Frontend (Vercel):** L'interfaccia utente in React è distribuita tramite Vercel, che garantisce tempi di caricamento rapidi e gestisce in automatico il routing della Single Page Application.
- **Backend (Render):** Le API REST e la logica di business gestite da Spring Boot sono ospitate su Render come web service.
- **Database (PostgreSQL):** I dati persistenti sono salvati su un'istanza PostgreSQL in cloud, connessa in modo sicuro al backend.

7.3 Configurazione e variabili d'ambiente

Per questioni di sicurezza, credenziali e link sensibili non sono tracciati nel repository. L'applicazione utilizza un sistema di variabili d'ambiente.

7.3.1 Lato Frontend (Vercel)

- **VITE_API_URL:** Punta all'URL pubblico del backend (<https://bugboard26.onrender.com>).
- *Nota per lo sviluppo locale:* L'app è provvista di una configurazione centralizzata (`src/config.js`). Se avviata in locale senza un file `.env`, l'app contatterà di default il server di produzione. Tramite un file `.env` locale è possibile deviare il traffico verso `http://localhost:8080`.

7.3.2 Lato Backend (Render)

- **ADMIN_PW**: Password dell'utente Admin fornito inizialmente.
- **DB_PASSWORD**: Password del Database.
- **DB_URL**: Stringa di connessione JDBC per il Database di produzione.
- **DB_USER**: Nome utente del Database.
- **FRONTEND_URL**: URL del client React autorizzato dal filtro CORS di Spring Security (<https://bug-board26-sigma.vercel.app>).
- *Nota per lo sviluppo locale*: Il file `application.properties` è configurato con valori di fallback. Se le variabili cloud non vengono trovate, Spring Boot si collegherà in automatico a un Database PostgreSQL locale (porta 5432) e aprirà il CORS alla porta 5173 di Vite.

8 Processo di sviluppo e qualità del codice

8.1 Versioning e collaborazione

Il processo di sviluppo è stato coordinato utilizzando il sistema version control di Git, con repository remoto ospitato su GitHub. Il team, composto da 3 membri, ha adottato un flusso di lavoro collaborativo mirato a minimizzare i conflitti. Sebbene gran parte dello sviluppo sia avvenuta in modo lineare, è stato scelto di adottare una strategia di branching per isolare le implementazioni architetturali più complesse. Nello specifico sono stati creati branch dedicati alla prima fase di integrazione e comunicazione tra Frontend e Backend, nonché per i test di configurazione legati al deployment in ambiente cloud. Questo approccio ha permesso di validare le configurazioni di rete e le variabili di ambiente in totale isolamento, effettuando il merge sul ramo principale (`main`) solo a seguito di esito positivo e stabile.

8.2 Continuous Integration e Continuous Deployment (CI/CD)

Il rilascio di nuove funzionalità è completamente automatizzato e gestito dai webhook di GitHub.

- Ogni operazione di `git push` o `merge` effettuata sul branch principale (`main`) innesca in simultanea i processi di build.
- **Vercel** intercetta le modifiche, ricompila il codice React e aggiorna istantaneamente il dominio di produzione.
- **Render** scarica i nuovi sorgenti Java, compila l'applicazione e riavvia il server Spring Boot senza richiedere alcun intervento manuale.

8.3 Report di qualità del codice

La validazione della qualità del codice sorgente (Backend) è stata effettuata tramite analisi statica utilizzando SonarQube. Il report della dashboard principale ha restituito risultati eccellenti, confermando la correttezza delle scelte architetturali e l'efficace applicazioni dei paradigmi Object-Oriented.

In particolare, il sistema ha ottenuto il massimo del rating (A) in tutte le categorie critiche:

- **Security e Reliability:** sono stati rilevati 0 bug, 0 vulnerabilità e 0 Security Hotspots, a dimostrazione di un codice sicuro e robusto.
- **Maintainability:** il sistema mantiene un Rating A. Il livello di Technical Debt è irrilevante e i pochi code smell residui (ridotti a poche decine grazie a costanti attività di refactoring) riguardano esclusivamente irregolarità sintattiche di lieve entità (e.g. typo). Inoltre, per mantenere l'analisi statica pulita da falsi positivi (evidenziati dall'IDE), è stato fatto uso mirato e consapevole dell'annotazione `@SuppressWarnings("NullableProblems")`.
- **Duplication:** il codice presenta un perfetto 0.0% di codice duplicato su oltre 3.200 righe analizzate, evidenziando l'uso rigoroso del riuso dei componenti e della modularità.
- **Test Coverage:** in piena conformità con le specifiche di progetto, la percentuale si attesta attorno al 21% riflettendo un approccio di collaudo mirato. Infatti, i test sono stati concentrati esclusivamente sui metodi non banali e critici della logica di business (livello Service), aventi almeno due parametri.

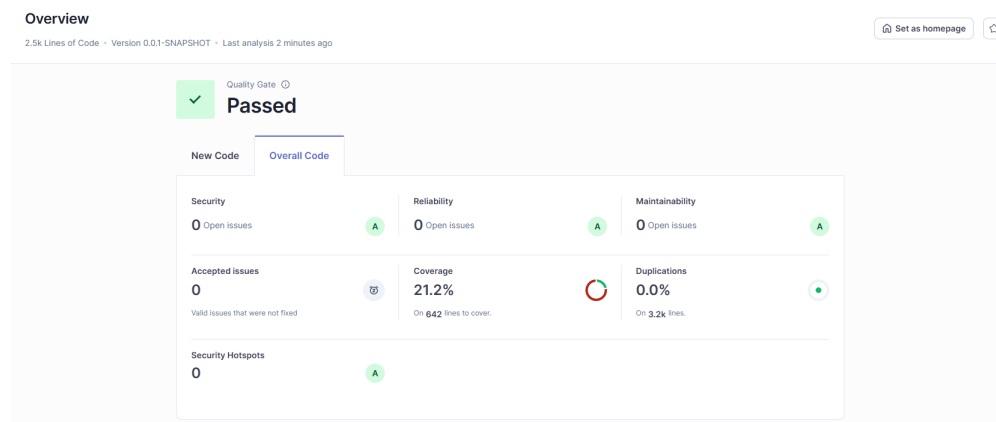


Figure 21: report di SonarQube

8.4 Artefatto software e repository link

Tutti gli artefatti software prodotti durante il ciclo di vita del progetto, comprensivi di codice sorgente e file di configurazione, sono centralizzati e accessibili tramite il repository GitHub del team, reperibile al seguente indirizzo: <https://github.com/il-Drucoder/BugBoard26>.

Per quanto riguarda il Backend, il progetto include il file `pom.xml` per la gestione automatizzata delle dipendenze e della build tramite Maven, affiancato dal `Dockerfile` per la containerizzazione dell'applicativo.

Per il Frontend invece, è fornito il file `package.json` contenente le librerie necessarie (e.g. React e Tailwind) e gli script per l'avvio della compilazione.

9 Testing

In questa sezione vengono descritte le tecniche e i metodi di testing applicati.

9.1 Testing di unità (Backend)

In conformità con le specifiche di progetto, la fase di testing non ha avuto come obiettivo il raggiungimento di una Code Coverage globale, è invece concentrata sull'analisi rigorosa di alcune delle funzionalità più critiche del sistema:

- Creazione Issue
- Assegnazione di Issue
- Cambio Password

Questi tre metodi fanno parte di classi *stateless*, verranno quindi testate in *isolation testing*: verranno impostati degli stub per i metodi critici da cui dipendono utilizzando la libreria Mockito, e verificheremo che queste funzioni vengano chiamate solo in determinate condizioni, corrispondenti a una combinazione di classi d'equivalenza degli input.

L'obiettivo del testing è stato, quindi, raggiungere un'alta percentuale di *CovLOC*, *condition coverage* e di *decision coverage* nei tre metodi scelti.

Vale la pena menzionare che la struttura delle classi di testing specchia le classi testate: la classe contiene i mock object della classe da testare, e i metodi relativi alla medesima funzione da testare sono stati raggruppati in una classe interna, che è invece responsabile di dichiarare i mock objects di cui ha bisogno e i dummy objects. Nella funzione di setup prima di ogni test vengono dati valori di default validi ai dummy objects, se il test sta verificando, invece, che la funzione si comporti correttamente quando riceve degli input invalidi, dovrà modificarli di conseguenza.

È stato adottato questo metodo per accertarsi che il motivo del comportamento specifico di una funzione fosse sempre controllato (i.e. una throw corrisponde effettivamente al valore invalido scelto, e non ad altri errori di setup).

9.1.1 Segnalazione di una issue (createIssue)

Il metodo `createIssue` della classe `IssueService` ha il compito di salvare le issue segnalate dagli utenti nel Database, è al cuore dell'applicazione ed è quindi naturale volersi accertare che svolga la sua funzione correttamente.

Codice del test (JUnit):

```
1  @Nested
2  class createIssueTest {
3
4      String dummyIssueTitle;
5      ReportIssueRequest dummyRequest;
6
7      Long dummyCurrentUserId;
8      User dummyCurrentUser;
9
10     @Mock
11     private MultipartFile dummyFile;
12     byte[] dummyBytes;
13
14     @BeforeEach
15     void setUpObjects() {
16
17         dummyCurrentUserId = 1L;
18
19         dummyIssueTitle = "Dummy Title";
20
21         dummyRequest = ReportIssueRequest.builder()
22             .title(dummyIssueTitle)
23             .description("Dummy description")
24             .type(IssueType.BUG.name())
25             .priority(false)
26             .build();
27
28         dummyCurrentUser = User.builder()
29             .id(dummyCurrentUserId)
30             .role(UserRole.USER)
31             .build();
32
33         dummyBytes = new byte[]{0, 0, 0};
34     }
35
36     @Test
37     void testNeedPermissionToCreateIssue() {
38         dummyCurrentUser.setRole(UserRole.LURKER);
39
40         //Mock setup
41         when(userRepository.findById(dummyCurrentUserId))
42             .thenReturn(Optional.of(dummyCurrentUser));
43
44         //Call to method
45         assertThrows(AccessDeniedException.class,
46             () -> issueService.createIssue(dummyRequest,
47                 dummyCurrentUserId, null),
48             "Did not throw AccessDeniedException");
49
50         //Verification
```

```

50     verify(issueRepository, times(0)).save(any());
51     verify(historyService, times(0)).createHistory(any(), any());
52 }
53
54 @Test
55 void testSavesAndCreatesHistory() throws IOException {
56
57     //Mock setup
58     when(userRepository.findById(dummyCurrentUserId))
59         .thenReturn(Optional.of(dummyCurrentUser));
60
61     when(tagRepository.findByNameIn(any())).thenReturn(null);
62
63     when(dummyFile.isEmpty()).thenReturn(false);
64     when(dummyFile.getContentType()).thenReturn("png");
65     when(dummyFile.getBytes()).thenReturn(dummyBytes);
66
67     when(issueRepository.save(any(Issue.class))).then(i ->
68         i.getArguments()[0]);
69
70     doNothing().when(historyService)
71         .createHistory(any(HistoryRequest.class),
72             eq(dummyCurrentUserId));
73
74     //Call to method
75     IssueDetailsResponse result =
76         issueService.createIssue(dummyRequest, dummyCurrentUserId,
77             dummyFile);
78
79     //Verification
80     assertNotNull(result, "Result does not exist");
81     assertEquals(dummyIssueTitle, result.getTitle(),
82         result.getTitle());
83
84     verify(issueRepository).save(any(Issue.class));
85     verify(historyService)
86         .createHistory(any(HistoryRequest.class),
87             eq(dummyCurrentUserId));
88
89     assertNotNull(result.getImage(), "Result does not have Image");
90     assertEquals(dummyBytes, result.getImage().getRawImage(),
91         "Result image does not match dummy image");
92
93 }
94
95 @Test
96 void testNonExistentUserThrowsAndDoesntSave() {
97
98     //Mock setup
99     when(userRepository.findById(dummyCurrentUserId))
100         .thenReturn(Optional.empty());
101
102     //Call to method to test
103     assertThrows(EntityNotFoundException.class,
104         () -> issueService.createIssue(dummyRequest,
105             dummyCurrentUserId, null),
106         "Did not throw EntityNotFoundException");
107
108 }

```

```

99
100     //Verification
101     verify(issueRepository, times(0)).save(any());
102     verify(historyService, times(0)).createHistory(any(), any());
103 }
104
105 @Test
106 void testMissingFileSaves() {
107
108     //Mock setup
109     when(userRepository.findById(dummyCurrentUserId))
110     .thenReturn(Optional.of(dummyCurrentUser));
111
112     when(tagRepository.findByNameIn(any())).thenReturn(null);
113
114     when(dummyFile.isEmpty()).thenReturn(true);
115
116     when(issueRepository.save(any(Issue.class))).then(i ->
117         i.getArguments()[0]);
118
119     doNothing().when(historyService)
120     .createHistory(any(HistoryRequest.class),
121         eq(dummyCurrentUserId));
122
123     //Call to method to test
124     IssueDetailsResponse result =
125         issueService.createIssue(dummyRequest, dummyCurrentUserId,
126             dummyFile);
127
128     //Verification
129     assertNotNull(result, "Result does not exist");
130
131     verify(issueRepository).save(any(Issue.class));
132     verify(historyService)
133     .createHistory(any(HistoryRequest.class),
134         eq(dummyCurrentUserId));
135
136     assertNull(result.getImage(), "Result has Image");
137 }
138
139 @Test
140 void testEmptyFileSaves() {
141
142     //Mock setup
143     when(userRepository.findById(dummyCurrentUserId))
144     .thenReturn(Optional.of(dummyCurrentUser));
145
146     when(tagRepository.findByNameIn(any())).thenReturn(null);
147
148     when(issueRepository.save(any(Issue.class))).then(i ->
149         i.getArguments()[0]);
150
151     doNothing().when(historyService)
152     .createHistory(any(HistoryRequest.class),
153         eq(dummyCurrentUserId));
154
155     //Call to method to test

```

```

149     IssueDetailsResponse result =
150         issueService.createIssue(dummyRequest, dummyCurrentUserId,
151             null);
152
153     //Verification
154     assertNotNull(result, "Result does not exist");
155
156     verify(issueRepository).save(any(Issue.class));
157     verify(historyService)
158         .createHistory(any(HistoryRequest.class),
159             eq(dummyCurrentUserId));
160
161     assertNull(result.getImage(), "Result has Image");
162 }
163
164 @Test
165 void testFailedFileReadThrowsExceptionAndDoesntSave() throws
166     IOException {
167
168     //Mock setup
169     when(userRepository.findById(dummyCurrentUserId))
170         .thenReturn(Optional.of(dummyCurrentUser));
171
172     when(tagRepository.findByNameIn(any())).thenReturn(null);
173
174     when(dummyFile.isEmpty()).thenReturn(false);
175     when(dummyFile.getContentType()).thenReturn("png");
176     when(dummyFile.getBytes()).thenThrow(IOException.class);
177
178     //Call to method to test
179     assertThrows(UncheckedIOException.class,
180         () -> issueService.createIssue(dummyRequest,
181             dummyCurrentUserId, dummyFile),
182         "Did not throw UncheckedIOException");
183
184     //Verification
185     verify(issueRepository, times(0)).save(any());
186     verify(historyService, times(0)).createHistory(any(), any());
187 }
188 }

```

```

40 alessa... @Transactional
41 alessa... public IssueDetailsResponse createIssue(ReportIssueRequest reportIssueRequest, Long currentUserId, MultipartFile file) {
42 alessa...     User reportingUser = findUserOrThrow(currentUserId);
43 sino04...
44 alessa...     if(!reportingUser.getRole().hasPermission(RolePermission.REPORT_ISSUE)){
45         throw new AccessDeniedException("L'utente è abilitato a segnalare issue");
46     }
47
48 sino04...     Set<Tag> tags = tagRepository.findByNameIn(reportIssueRequest.getTags());
49 sino04...
50 alessa...     IssueImage image = null;
51
52 sino04...     if (file != null && !file.isEmpty()) {
53         String extension = Objects.requireNonNull(file.getContentType()).substring(Objects.requireNonNull(file
54             .getContentType()).lastIndexOf('/') + 1);
55         String storedName = UUID.randomUUID() + "." + extension;
56
57         try{
58             image = IssueImage.builder()
59                 .rawImage(file.getBytes())
60                 .name(storedName).build();
61         } catch (IOException e) {
62             throw new UncheckedIOException("Could not save file", e);
63         }
64
65 sino04...     Issue issue = Issue.builder()
66         .title(reportIssueRequest.getTitle())
67         .description(reportIssueRequest.getDescription())
68         .type(IssueType.valueOf(reportIssueRequest.getType()))
69         .status(IssueStatus.TODO)
70         .priority(reportIssueRequest.getPriority())
71         .tags(tags)
72         .image(image)
73         .creationDate(LocalDate.now(ZoneId.systemDefault()))
74         .lastModifiedDate(LocalDate.now(ZoneId.systemDefault()))
75         .reportingUser(reportingUser)
76         .assignedUser(null).build();
77
78 sino04...     Issue savedIssue = issueRepository.save(issue);
79
80 alessa...     HistoryRequest historyRequest = new HistoryRequest(savedIssue.getId(), "ha segnalato la issue");
81 alessa...     historyService.createHistory(historyRequest, currentUserId);
82 sino04...
83 alessa...     return convertModelToIssueDetails(savedIssue);
84 sino04... }

```

Figure 22: coverage di createIssue

9.1.2 Assegnazione di una issue ad uno User (assignUserToIssue)

: il metodo `assignIssueToUser` della classe `IssueService` permette agli utenti Admin di assegnare le issue agli User, dividendo il lavoro dell'azienda e tenendo traccia dei compiti svolti dai singoli dipendenti.

Codice del test (JUnit):

```
1  @Nested
2  class assignUserToIssueTest {
3
4      Long dummyIssueId;
5      Long dummyUserId;
6      Long dummyCurrentUserId;
7
8      User dummyUser;
9
10     User dummyCurrentUser;
11
12     Issue dummyIssue;
13
14     @BeforeEach
15     void setUpObject() {
16
17         dummyIssueId = 1L;
18         dummyUserId = 22L;
19         dummyCurrentUserId = 333L;
20
21         dummyUser = User.builder()
22             .id(dummyUserId)
23             .role(UserRole.USER)
24             .build();
25
26         dummyCurrentUser = User.builder()
27             .role(UserRole.ADMIN)
28             .build();
29
30         dummyIssue = Issue.builder()
31             .type(IssueType.BUG)
32             .status(IssueStatus.TODO)
33             .reportingUser(User.builder()
34                 .id(0L)
35                 .build())
36             .build();
37     }
38
39     //Users not existing have already been tested in the
40     //createIssueTest class
41     //((Check coverage of FindUserOrThrow))
42
43     @Test
44     void testNonExistentIssueThrowsAndDoesntSave() {
45
46         //Mock setup
47         when(issueRepository.findById(dummyIssueId))
48             .thenReturn(Optional.empty());
```

```

49 //Call to method to test
50 assertThrows(EntityNotFoundException.class,
51             () -> issueService
52                 .assignUserToIssue(dummyIssueId, null, null));
53
54 //Verification
55 verify(issueRepository, times(0)).save(any());
56 verify(historyService, times(0)).createHistory(any(), any());
57 }
58
59 @Test
60 void testIssueIsAlreadyAssignedToOtherUserThrowsAndDoesntUpdate() {
61
62     dummyUser.setId(dummyUserId + 10); //make sure that it is
63         different from dummyUserId
64     dummyIssue.setAssignedUser(dummyUser);
65
66     //Mock setup
67     when(issueRepository.findById(dummyIssueId))
68         .thenReturn(Optional.of(dummyIssue));
69
70     //Call to method to test
71     assertThrows(IllegalStateException.class,
72                 () -> issueService.assignUserToIssue(dummyIssueId,
73                                                         dummyUserId, null));
74
75     //Verification
76     verify(issueRepository, times(0)).save(any());
77     verify(historyService, times(0)).createHistory(any(), any());
78 }
79
80 @Test
81 void testIssueIsAlreadyAssignedToSameUserReturnsAndDoesntUpdate() {
82
83     dummyIssue.setAssignedUser(dummyUser);
84
85     //Mock setup
86     when(issueRepository.findById(dummyIssueId))
87         .thenReturn(Optional.of(dummyIssue));
88
89     //Call to method to test
90     IssueDetailsResponse result =
91         issueService.assignUserToIssue(dummyIssueId, dummyUserId,
92                                         null);
93
94     //Verification
95     assertNotNull(result, "Result doesn't exist");
96     assertEquals(result.getAssignedUserId(), dummyUserId, "Ids do
97         not match");
98
99     verify(issueRepository, times(0)).save(any());
100    verify(historyService, times(0)).createHistory(any(), any());
101 }
102
103 @Test
104 void testIssueIsAlreadyProgressedThrowsAndDoesntUpdate() {

```

```

101     dummyIssue.setStatus(IssueStatus.INPROGRESS);
102
103     //Mock setup
104     when(issueRepository.findById(dummyIssueId))
105     .thenReturn(Optional.of(dummyIssue));
106
107     //Call to method to test
108     assertThrows(IllegalStateException.class,
109         () -> issueService.assignUserToIssue(dummyIssueId,
110             dummyUserId, null));
111
112     //Verification
113     verify(issueRepository, times(0)).save(any());
114     verify(historyService, times(0)).createHistory(any(), any());
115 }
116
117 @Test
118 void testCurrentUserCantAssignIssueThrowsAndDoesntUpdate() {
119
120     dummyCurrentUser.setRole(UserRole.USER);
121
122     //Mock setup
123     when(issueRepository.findById(dummyIssueId))
124     .thenReturn(Optional.of(dummyIssue));
125
126     when(userRepository.findById(dummyCurrentUserId))
127     .thenReturn(Optional.of(dummyCurrentUser));
128
129     //Call to method to test
130     assertThrows(AccessDeniedException.class,
131         () -> issueService.assignUserToIssue(dummyIssueId,
132             dummyUserId, dummyCurrentUserId));
133
134     //Verification
135     verify(issueRepository, times(0)).save(any());
136     verify(historyService, times(0)).createHistory(any(), any());
137 }
138
139 @Test
140 void testUserCantBeAssignedIssueThrowsAndDoesntUpdate() {
141
142     dummyUser.setRole(UserRole.LURKER);
143
144     //Mock setup
145     when(issueRepository.findById(dummyIssueId))
146     .thenReturn(Optional.of(dummyIssue));
147
148     when(userRepository.findById(dummyCurrentUserId))
149     .thenReturn(Optional.of(dummyCurrentUser));
150     when(userRepository.findById(dummyUserId))
151     .thenReturn(Optional.of(dummyUser));
152
153     //Call to method to test
154     assertThrows(AccessDeniedException.class,
155         () -> issueService.assignUserToIssue(dummyIssueId,
156             dummyUserId, dummyCurrentUserId));

```

```

155     //Verification
156     verify(issueRepository, times(0)).save(any());
157     verify(historyService, times(0)).createHistory(any(), any());
158 }
159
160 @Test
161 void testAssigningUserSavesAndUpdatesHistory() {
162
163     //Mock setup
164     when(issueRepository.findById(dummyIssueId))
165         .thenReturn(Optional.of(dummyIssue));
166
167     when(userRepository.findById(dummyCurrentUserId))
168         .thenReturn(Optional.of(dummyCurrentUser));
169     when(userRepository.findById(dummyUserId))
170         .thenReturn(Optional.of(dummyUser));
171
172     doNothing().when(historyService)
173         .createHistory(any(HistoryRequest.class),
174             eq(dummyCurrentUserId));
175
176     when(issueRepository.save(any(Issue.class))).then(i ->
177         i.getArguments()[0]);
178
179     //Call to method to test
180     IssueDetailsResponse result =
181         issueService.assignUserToIssue(dummyIssueId, dummyUserId,
182             dummyCurrentUserId);
183
184     //Verification
185     assertNotNull(result, "Result doesn't exist");
186     assertEquals(result.getAssignedUserId(), dummyUserId, "Ids do
187         not match");
188
189     verify(issueRepository).save(any(Issue.class));
190     verify(historyService).
191         createHistory(any(HistoryRequest.class),
192             eq(dummyCurrentUserId));
193 }
194 }

```

```

150      @Transactional
151      public IssueDetailsResponse assignUserToIssue(Long issueId, Long userId, Long currentUserId) {
152          Issue issue = findIssueOrThrow(issueId);
153
154          if (issue.getAssignedUser() != null) {
155              if (issue.getAssignedUser().getId().equals(userId)) {
156                  return convertModelToIssueDetails(issue);
157              }
158
159              throw new IllegalStateException("Attenzione, questa issue è già stata assegnata all'utente: " + issue
160                  .getAssignedUser().getUsername());
161
162              if (!issue.getStatus().equals(IssueStatus.TODO)) {
163                  throw new IllegalStateException("Impossibile assegnare questa issue, si trova già nello stato: " + issue
164                      .getStatus().name());
165              }
166
167              User currentUser = findUserOrThrow(currentUserId);
168              if(!currentUser.getRole().hasPermission(RolePermission.ASSIGN_ISSUE)){
169                  throw new AccessDeniedException("L'utente non è abilitato ad assegnare issue");
170              }
171
172              User assignedUser = findUserOrThrow(userId);
173              if(!assignedUser.getRole().hasPermission(RolePermission.BE_ASSIGNED_TO_ISSUE)){
174                  throw new AccessDeniedException("L'utente non può essere assegnato ad una issue");
175              }
176
177              issue.setAssignedUser(assignedUser);
178
179              Issue savedIssue = issueRepository.save(issue);
180
181              HistoryRequest historyRequest = new HistoryRequest(savedIssue.getId(), "ha assegnato la issue a " + savedIssue
182                  .getAssignedUser().getUsername());
183              historyService.createHistory(historyRequest, currentUserId);
184
185              return convertModelToIssueDetails(savedIssue);
186          }
187      }

```

Figure 23: coverage di `assignUserToIssue`

9.1.3 Cambio password utente (`changeUserPassword`)

Il metodo `changeUserPassword` di `UserService` consente a qualsiasi utente di cambiare password, un metodo fondamentale considerando che gli account vengono forniti da utenti Admin con password da loro scelte, è quindi necessario verificare che svolga il suo compito per garantire la sicurezza degli utenti della piattaforma.

Codice del test (JUnit):

```
1  @Nested
2  class ChangeUserPasswordTest {
3
4      Long dummyCurrentUserId;
5      User dummyCurrentUser;
6
7      String dummyOldPassword;
8      String dummyNewPassword;
9      ChangePasswordUserRequest dummyRequest;
10
11     @BeforeEach
12     void setUpObjects() {
13         dummyCurrentUserId = 1L;
14         dummyCurrentUser = User.builder()
15             .id(dummyCurrentUserId)
16             .role(UserRole.USER)
17             .build();
18
19         dummyOldPassword = "abc";
20         dummyNewPassword = "xyz";
21         dummyRequest = ChangePasswordUserRequest.builder()
22             .currentRawPassword(dummyOldPassword)
23             .newRawPassword(dummyNewPassword)
24             .build();
25     }
26
27     @Test
28     void testNonExistentUserThrowsAndDoesntSave() {
29
30         //Mock setup
31         when(userRepository.findById(dummyCurrentUserId))
32             .thenReturn(Optional.empty());
33
34         //Call to method to test
35         assertThrows(EntityNotFoundException.class,
36             () -> userService
37                 .changeUserPassword(dummyCurrentUserId, dummyRequest),
38             "Did not throw EntityNotFoundException");
39
40         //Verification
41         verify(userRepository, times(0)).save(any());
42     }
43
44     @Test
45     void testWrongOldPasswordThrowsAndDoesntSave() {
46
47         //Mock setup
48         when(userRepository.findById(dummyCurrentUserId))
49             .thenReturn(Optional.of(dummyCurrentUser));
50
51         when(passwordEncoder.matches(eq(dummyOldPassword),
52             any())) .thenReturn(false);
53
54         //Call to method to test
55         assertThrows(ResponseStatusException.class,
56             () -> userService
```

```

56         .changeUserPassword(dummyCurrentUserId, dummyRequest),
57         "Did not throw ResponseStatusException");
58
59         //Verification
60         verify(userRepository, times(0)).save(any());
61     }
62 }
63
64 @Test
65 void testShouldSaveAndHaveDifferentPasswords() {
66
67     //Mock setup
68     when(userRepository.findById(dummyCurrentUserId))
69     .thenReturn(Optional.of(dummyCurrentUser));
70
71     when(passwordEncoder.matches(eq(dummyOldPassword),
72     any()))).thenReturn(true);
73
74     when(passwordEncoder.encode(dummyNewPassword))
75     .then(i -> i.getArguments()[0]);
76
77     when(userRepository.save(dummyCurrentUser))
78     .then(i -> i.getArguments()[0]);
79
80     //Call to method to test
81     UserResponse result = userService
82     .changeUserPassword(dummyCurrentUserId, dummyRequest);
83
84     //Verification
85     assertEquals(result.getId(), dummyCurrentUserId, "UserId has
86     changed");
87     verify(passwordEncoder).encode(dummyNewPassword);
88     verify(userRepository).save(dummyCurrentUser);
89     //we cannot check that the user's password has effectively
90     //changed (i.e. new is different from old)
91     //we will have to use intelliJ's coverage metrics to see that
92     setHashedPassword has been called
93 }
94 }

```

```

57  sino04...  @Transactional
58  sino04...  public UserResponse changeUserPassword(Long userId, ChangePasswordUserRequest userPasswords) {
59              User oldUser = findUserOrThrow(userId);
60
61              if (!passwordEncoder.matches(userPasswords.getCurrentRawPassword(), oldUser.getHashedPassword())) {
62                  throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "La password corrente non è corretta");
63              }
64
65              oldUser.setHashedPassword(passwordEncoder.encode(userPasswords.getNewRawPassword()));
66
67              User savedUser = userRepository.save(oldUser);
68
69              return convertModelToResponse(savedUser);
70          }

```

Figure 24: coverage di `changeUserPassword`

9.2 Usability Testing

Come supporto ai test automatizzati di unità sul Backend, è stata condotta una validazione del Frontend di tipo manuale. Questa scelta ha permesso sia di valutare la corretta integrazione tra i due livelli, ma anche di constatare l'usabilità generale dell'interfaccia utente (UI).

Le aree coinvolte dai test sono le seguenti:

- **Controllo di accessi e ruoli:** è stata verificata la corretta gestione (creazione e distruzione) del token JWT lato client. È stato verificato il blocco alle pagine non accessibili da utenti non autenticati, ed anche la corretta renderizzazione dei componenti grafici in base ai ruoli/permessi (e.g. il pulsante "chiudi issue" è visibile soltanto a coloro che ne hanno l'autorizzazione, nel nostro caso soltanto gli Admin).
- **Validazione form e feedback visivo:** sono stati simulati inserimenti di dati non validi (e.g. campi obbligatori assenti, formattazioni errate) per garantire che il Frontend intercetti gli errori e fornisca un feedback visivo immediato all'utente, ancor prima di interrogare il server.

9.2.1 Survey a persone esterne

Per accertarci dell'usabilità della piattaforma è stato chiesto a persone che non hanno partecipato in alcun modo allo sviluppo dell'applicazione per il loro feedback. Agli intervistati è stato descritto lo scopo generale della piattaforma e le responsabilità di un Admin (ruolo con cui hanno avuto accesso alla piattaforma).

Riportiamo qui sotto la tabella con le domande e le risposte degli utenti, dove applicabile è stata riportata anche la motivazione del giudizio dato.

Domanda	Risposta	Motivazione
Candidato 1		
La navigazione tra pagine era intuitiva?	Più sì che no	L'icona della home mi ha confuso credo dovrebbe trovarsi a sinistra
Lo scopo delle pagine che hai visitato era chiaro?	Sì	
In caso di errori, il sistema proponeva feedback chiaro?	Sì	
Hai trovato intuitivo il sistema di filtraggio e ordinamento?	Sì	
Hai trovato difficoltà nell'uso complessivo dell'applicazione?	Più no che sì	Dopo la creazione di una issue l'applicazione dovrebbe riportarti alla Home
Candidato 2		
La navigazione tra pagine era intuitiva?	Sì	
Lo scopo delle pagine che hai visitato era chiaro?	Più sì che no	La pagina di "issue assegnate" non mi ha fatto capire che fossero quelle assegnate a me
In caso di errori, il sistema proponeva feedback chiaro?	Sì	
Hai trovato intuitivo il sistema di filtraggio e ordinamento?	Sì	
Hai trovato difficoltà nell'uso complessivo dell'applicazione?	No	
Candidato 3		
La navigazione tra pagine era intuitiva?	Sì	
Lo scopo delle pagine che hai visitato era chiaro?	Più sì che no	Ho avuto dei piccoli dubbi sulla funzione delle pagine
In caso di errori, il sistema proponeva feedback chiaro?	Sì	
Hai trovato intuitivo il sistema di filtraggio e ordinamento?	Sì	
Hai trovato difficoltà nell'uso complessivo dell'applicazione?	No	

Table 1: Risultati dei test di usabilità per singolo candidato

9.2.2 Modifiche effettuate

A seguito del riscontro dato dagli intervistati è stato considerato opportuno effettuare le seguenti modifiche all'interfaccia utente:

- Riposizionamento tasto Home sul lato sinistro, accanto al menu hamburger.
- Modificato l'indirizzamento verso la Home dopo aver creato una issue.
- Aggiunta piccola descrizione in "Issue assegnate" e "Issue segnalate", così da far capire meglio all'utente cosa sta visualizzando.